Verification Continuum

HAPS® Prototyping Instrumentor User Guide

Version V-2025.06-3, April 2026

SYNOPSYS®

Copyright and Proprietary Information Notice

© 2026 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys, Ansys, the Synopsys and Ansys logos, and other Synopsys trademarks are available at https://www.synopsys.com/company/legal/trademarks-brands.html.
Other company or product names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

Feedback

Contents

1. **Preface** 6
   - Related Documentation 6

2. **The Design Instrumentation and Debug Flow** 8
   - Instrumentation Process 10
   - Launching the Instrumentor 10
   - Instrumentor Windows and Views 12
   - Configuring Instrumentor Preferences 14

3. **Using the Instrumentor** 18
   - Instrumenting a Design with Unified Compile 18
     - Setting Instrumentation Limits for UC Designs 19
     - Specifying Post-Compile Instrumentation for UC Designs 20
   - Using SystemVerilog Assertions 21
     - Adding SVAs for Instrumentation 21
     - Detecting Assertion Failures with a HAPS System Reset 23
     - Debugging Designs with SVAs 24
   - Using $dumpvars 25
     - Adding $dumpvars for Instrumentation 26
     - Using Wildcard Characters in $dumpvars 28
     - Specifying $dumpvars for RTL Hierarchies 29

4. **Setting up the IDC and IICE** 31
   - Configuring an IDC File 31
     - Modifying the IDC File 32
   - Configuring IICE Units 33
     - Configuring IICE Units 34
     - Configuring Individual IICE Parameters 35
     - Configuring IICE Sampler Parameters for an IICE 37

Configuring IICE Sample Clock 39
Guidelines for Selecting a Sample Clock 43
Configuring IICE Controller and Other Options 44
Setting up Simple Triggers 47
Setting up Complex Counter Triggers 47
Setting up State Machine Triggers 49
Setting up an IICE for Cross-Triggering 52
Setting up RTL Cross Triggers 54
Configuring Multiple IICE Units 55
Using Multiple IICEs for Multiple Instrumentation Modes 55
Examples of $dumpvars Labels for Multi-IICE Grouping 56

5. HAPS Deep Trace Debug 60

Specifying DTD Buffers for HAPS Multi-FPGA Designs 60

1

Preface

This section contains the following topics:

- Related Documentation

About this Guide

The *HAPS Prototyping Instrumentor User Guide* describes the functionality of the Instrumentor and how to use the Instrumentor features.

This document is intended for users who are familiar with the HAPS ProtoCompiler product range.

Contents of this Guide

The *HAPS Prototyping Instrumentor User Guide* have the following chapters:

- The Design Instrumentation and Debug Flow
- Using the Instrumentor
- Setting up the IDC and IICE
- HAPS Deep Trace Debug

Related Documentation

The following product documents are included within the HAPS Prototyping tools. Unless otherwise specified, all documents can be accessed through the help system and as PDF documents:

- *HAPS® Prototyping User Guide*
- *HAPS® Prototyping Reference Manual*
- *HAPS® Prototyping Command Reference Manual*
- *HAPS® Prototyping Compiler and Mapper Guide*
- *HAPS® Prototyping Debugging Environment Reference Manual* (HAPS-100 only)
- *HAPS® Prototyping Debugger User Guide* (HAPS-100 only)

- *HAPS® Prototyping Instrumentor User Guide*
- *HAPS® Prototyping Message Reference Manual*
- *HAPS® Prototyping hRun User Guide* (HAPS-200 only)
- *HAPS® Prototyping Limited Customer Availability (LCA) Features Guide*
- *System Configuration Software Handbook* (HAPS-100 only)
- Free and Open-Source Software (FOSS) information is in PDF format and is located in the install_dir/doc directory.

Additional documentation is available outside the software:

- HAPS hardware documentation is available on https://spdocs.synopsys.com/dow_retrieve/latest/home_public/haps.html.

2

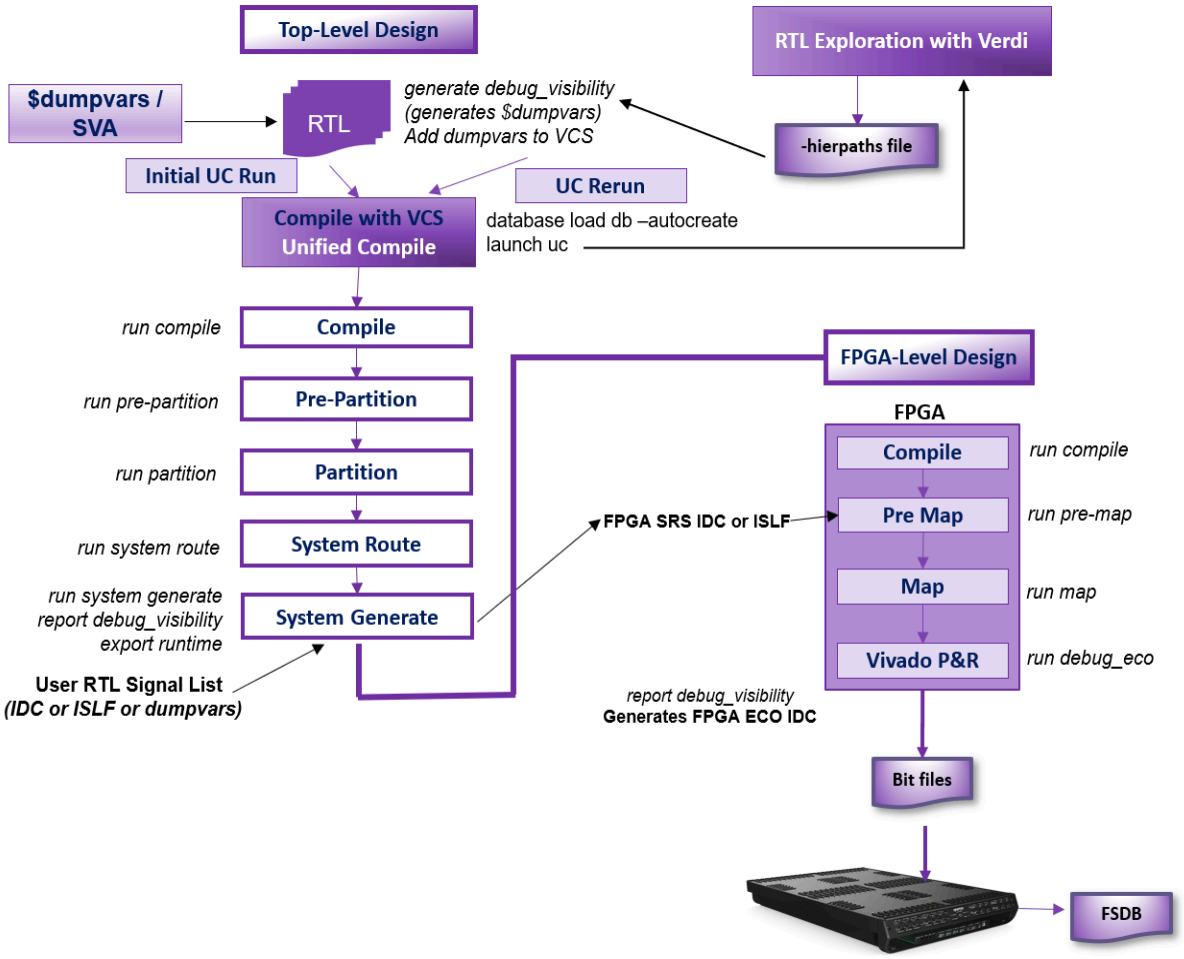
The Design Instrumentation and Debug Flow

The first step to prototyping with debug is to set up the design and mark signals for debugging. The process of marking signals for on-chip debugging later is called instrumentation. The instrumentor is the tool used to mark the signals.

Signal instrumentation information is stored as constraints in an *Identify Design Constraints (IDC)* file, which you add to the design. The figure shows the design instrumentation and debug flow.

Feedback

Figure 1 Instrumentation and Debug Flow



It is recommended to add instrumentation early in the cycle to reserve the logic resources needed. Incorporating instrumentation as part of the initial phase reduces turnaround time.

With the UC flow, use *$dumpvars* to specify instrumentation in the HDL. This is processed by the VCS unified compiler. The instrumentor is only used for post-compile instrumentation on an SRS (compiled) database. There is no pre-compile instrumentation through the instrumentor.

After compile, the design goes through synthesis and P&R. The bit files are generated and the design is programmed onto the hardware system. The instrumentation information is passed to the debugger. The design is run on the target hardware. The debugger verifies the design by monitoring the instrumented signals.

Instrumentation Process

Instrumentation consists of these steps:

1. Specify IICE parameters and HAPS settings.
2. Select signals to sample.
3. Export the files for debug. Optionally, include the original HDL source.

The IDC File

Signal information is stored as constraints in an *instrumentation design constraints* (`idc`) file. This is the basic instrumentation file. See Configuring an IDC File for more information.

The Intelligent In-Circuit Emulator (IICE)

The IICE™ (Intelligent In-Circuit Emulator) is a communication block that is added to the design for instrumentation and debug. Parameters for the IICE are specified in the IDC file. A design can have more than one IICE. See Configuring IICE Units for more information.

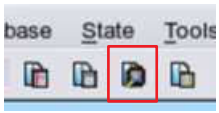
Launching the Instrumentor

Use the Instrumentor tool to identify signals for monitoring during debug. The instrumentor can be run from the command line or through a graphical user interface (GUI).

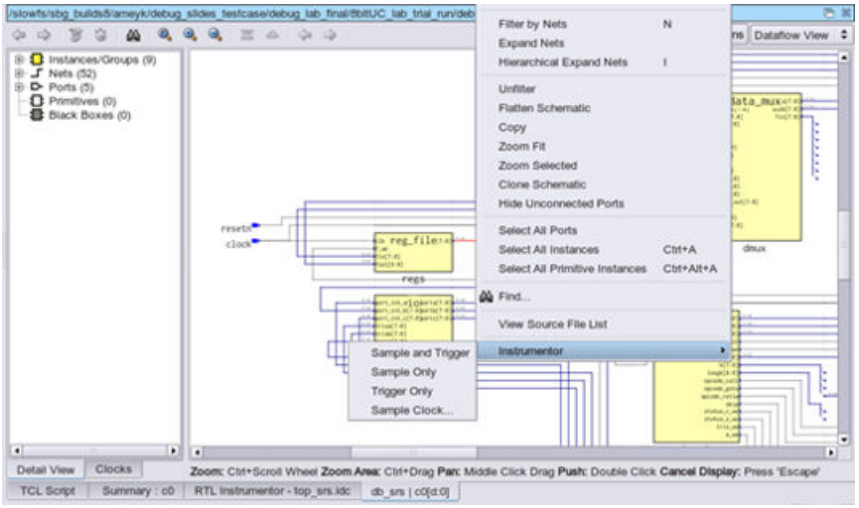
1. Open the Instrumentor.
   - To launch the instrumentor from the command line, specify this command at pre-map state during FPGA synthesis:

     ```
     edit idc -mode gui <fileName>.idc
     ```

     You can define the SRS-based IDC file.
   - To start the instrumentor from the GUI, click the icon in the prototyping tool:

     The Edit Instrumentation dialog displays. Browse and select the IDC file. Select the mode as **GUI** or **Text**. Click **Open**.
2. In the GUI that opens, select the signals and set sample and trigger instrumentation for them.



3. Follow these guidelines when editing the IDC information in the file.
   - Make sure the hierarchical signal path starts with the top-level module.
   - Use a period (.) not a slash (/), as the path hierarchy separator.
   - Specify signals in the generate scope with square brackets [] instead of brackets (), because there is no need to specify the range for a signal in IDC.

     Example:

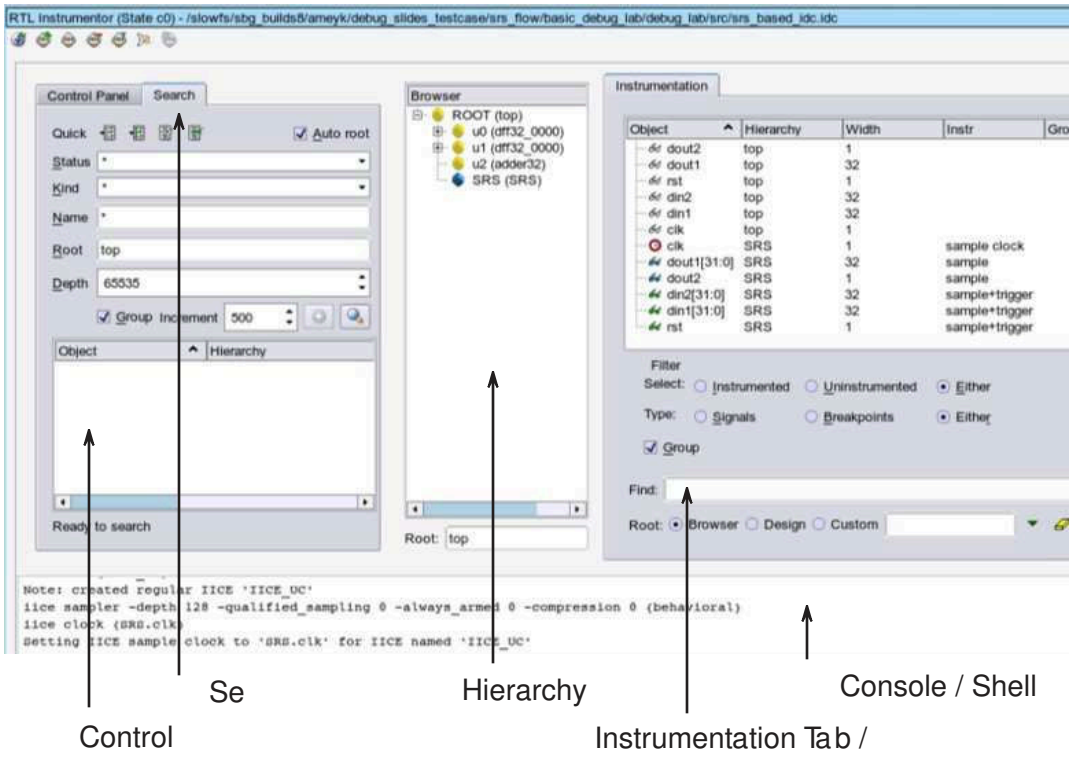
     ```
     iice clock -iice {IICE_UC} -edge positive {top.clock_c}
     signals add -iice {IICE_UC} -silent -trigger -sample {top.i1.i2.s1}
     ```

4. Run through the instrumentation and debug flow described in the *HAPS Prototyping Debug Guide*, and use the Verdi interface to view and analyze the probe signals.

Feedback

Instrumentor Windows and Views

The Graphical User Interface (GUI) has these areas:



- Control Panel
- Hierarchy Browser
- Instrumentation Tab

Control Panel

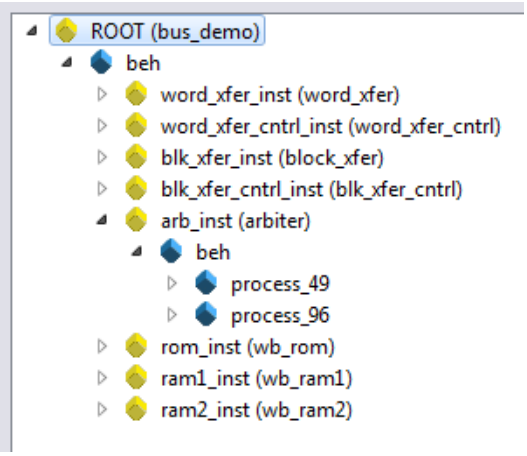
The **Control Panel** tab describes the current status of the instrumented design. Note that some entries are dependent on the IICE sampler buffer type selected.

Hierarchy Browser

The hierarchy browser shows a graphical representation of the design hierarchy. At the top of the browser is the ROOT node It represents the top-level entity or module of your design. For VHDL designs, the first level below the ROOT is the architecture of the top-level entity. The next level shows the entities instantiated at the top level. For Verilog design, the level below ROOT shows the modules instantiated at the top level.

Double-clicking an entry opens the entity/module instance and displays the hierarchy below that instance. Lower hierarchical levels show instantiations, *case* statements, *if* statements, functional operators, and other statements.

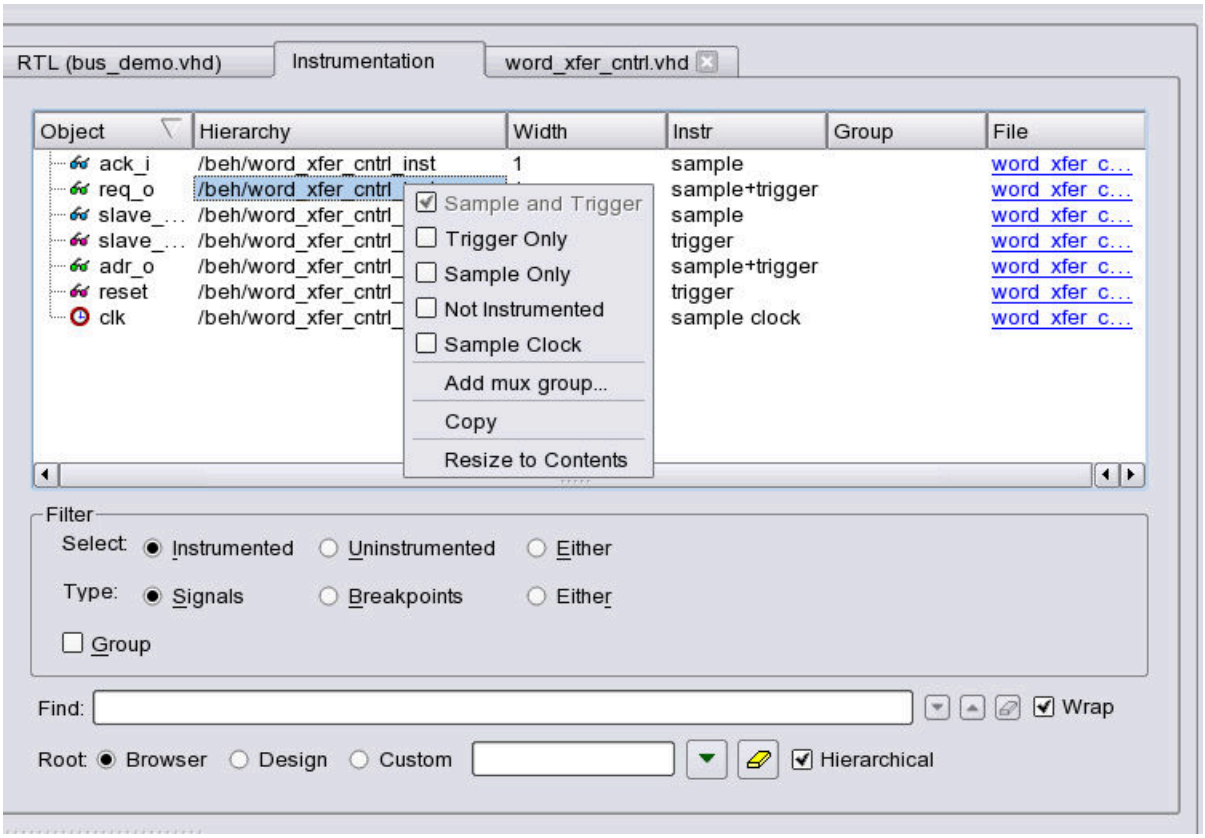
Single clicking any element in the hierarchy browser causes the associated HDL code to be visible in the **RTL** tab display.



Certain modules cannot be instrumented. For example, the contents of black-box modules, which are represented by a black icon in the browser.

Instrumentation Tab

The **Instrumentation** tab is also called the Signals view. The entries can be modified by selecting the entry and assigning a new value from the popup menu.

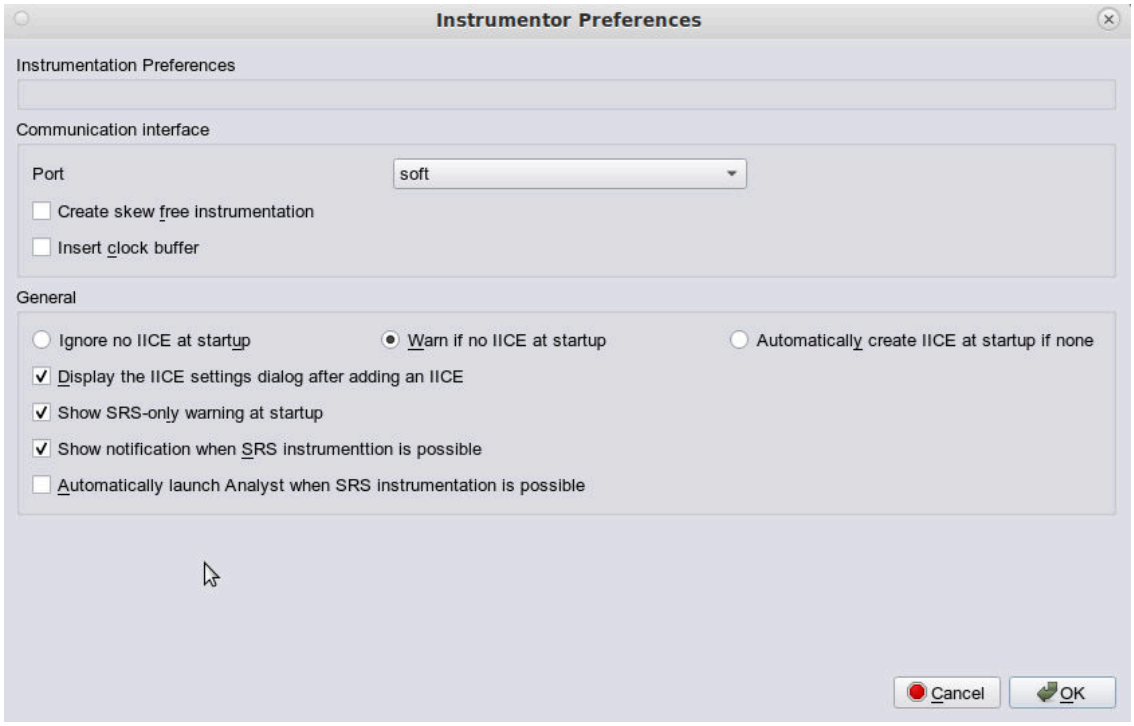


Configuring Instrumentor Preferences

Configure parameters common to all IICE units in a multi-IICE design. For information about setting parameters for individual IICE units, see Configuring Individual IICE Parameters.

Feedback

1. Select **Instrumentor > Instrumentor Preferences** from the top menu bar or click the **Communication Interface** entry in the **Control Panel** to open the **Instrumentor Preferences** dialog box.

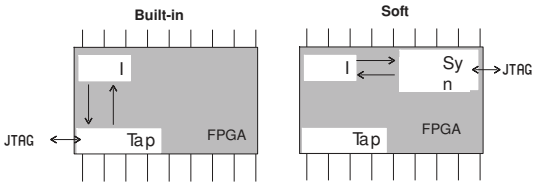


2. Configure the following parameters:

Table 1 Instrumentor Preferences

Parameter	Description
Communication Interface	

Feedback

Parameter	Description
Port	Specifies the type of connection used to communicate with the on-chip IICE. The connection types available from the drop-down menu are: ◦ **builtin**: Indicates that the IICE is connected to the JTAG Tap controller available on the target device.  ◦ **soft**: Indicates that the IICE is connected to the Synopsys Tap controller available on the target device. The Synopsys FPGA Tap controller uses more resources because it is implemented in user logic. It requires four user I/O pins to connect to the communication cable. ◦ **umrbus**: Indicates that the IICE is connected to the target device through the UMRBus.
Create skew free instrumentation	Incorporates skew-free hardware consisting of primary-secondary flip-flops on the JTAG chain. Allows the instrumentation logic to operate without requiring an additional global clock buffer resource for the JTAG clock. If this option is enabled: ◦ When global clock resources are not available for the JTAG clock, the IICE is built using skew-free hardware. This prevents clock skew from affecting the logic. ◦ The JTAG clock is not defined as requiring global clock resources.
Insert clock buffer	When using the JTAG interface, enabling the check box automatically inserts two global clock buffers to the debug IP.
General	
Ignore no IICE at startup	Select this checkbox to take no action when opening the instrumentor and there is no IICE defined.
Warn if no IICE at startup	Select this option to issue a warning when opening the instrumentor and there is no IICE defined.
Automatically create IICE at startup if none	Select this checkbox to create an IICE when opening the instrumentor and there is no IICE defined.
Display the IICE settings dialog after adding an IICE	After creating an IICE (either in the **Add IICE** dialog box or automatically at startup), display the **IICE Settings** dialog box.

Parameter	Description
Show SRS-only warning at startup	When opening the instrumentor, issue a warning if only SRS instrumentation is possible.
Show notification when SRS instrumentation is possible	Select this checkbox to show a notification when SRS instrumentation is possible.
Automatically launch Analyst in SRS-only mode	When opening the instrumentor, automatically launch the HDL Analyst when SRS instrumentation is possible.

3

Using the Instrumentor

The Instrumentor:

- Defines the instrumentation for the HDL design
- Creates the instrumented HDL design
- Creates the associated IICE
- Creates the design database

For more information, see:

- Instrumenting a Design with Unified Compile
- Using SystemVerilog Assertions
- Using $dumpvars

Instrumenting a Design with Unified Compile

There are two ways to designate probe signals and instrument the RTL design with the UC flow: using SystemVerilog assertions and using *$dumpvars*. In addition to these RTL instrumentation techniques, you can add post-compile instrumentation.

The instrumented signals are managed through the Instrumentation-Debug Constraints file (IDC), which contains constraints for the instrumented signals as well as the Intelligent In-Circuit Emulator (IICE) which is the debug IP.

in the UC flow you cannot use the instrumentor to directly add instrumentation to the RTL. Instead, it is handled by the VCS tools. You can however, add post-compile or SRS instrumentation to the design after compile.

See the following for more information:

Instrumentation	Setting Instrumentation Limits for UC Designs Using SystemVerilog Assertions Using $dumpvars Specifying Post-Compile Instrumentation for UC Designs

At runtime, you can run the design on the hardware and debug the design. You can use the Synopsys Verdi software to check the waveforms from the instrumented signals against a golden version. You can also debug the design on hardware, but the details vary, depending on the hardware you use.

Setting Instrumentation Limits for UC Designs

You can set limits to the number of bits instrumented in a UC design, either globally or on a per-module level. Do this before running the prototyping flow. Global instrumentation limits are set in the UTF control file, and local limits are set in the RTL for the module. The two ways to specify instrumentation limits provide flexibility in a large design.

1. To set a global instrumentation limit, add this command to the UTF file:

   ```
   debug -dumpvars_maxbits <number>
   ```

   The UTF file provides the controls to run the VCS compiler as a part of unified compile.

   The instrumentation limit set in the UTF is a global command that sets a maximum limit on the number of signals that can be instrumented. For example, if it is set to 12, and more than 12 signals are instrumented, you get an error in the *uc.log* file:

   ```
   Error-[FS_MAXBITS_EXCEEDED] FSDB maxbits limit exceeded
   Total bit count (24) for nodes dumped by $dumpvars task call
   exceeds maxbits limit (12).
   ```

2. To set a local limit, add it to the module definition in the RTL.

   ```
   (* maxbits=< > *)
   ```

   The local limit overrides the global limit. The following example shows the instrumentation limit for the module set to 24:

   ```
    initial
   begin
   (* maxbits=24 *) $dumpvars (1, top.temp1,top.temp2,top.temp3);
   end
   ```

   If the listed signals are 8 bits each and the global limit is set to 12, the compiler would normally error out because 24 exceeds the maximum limit of 12. However, because the local limit on *$dumpvars* is set to 24, there is no compilation error.

Feedback

Specifying Post-Compile Instrumentation for UC Designs

With the UC flow, the prototyping software can only define RTL instrumentation through *$dumpvars* and SVAs. However, you can edit the instrumentation file (IDC) after the compile stage. Follow these guidelines.

1. Compile the UC design with the *launch uc* and *run compile* commands to generate a compiled database.

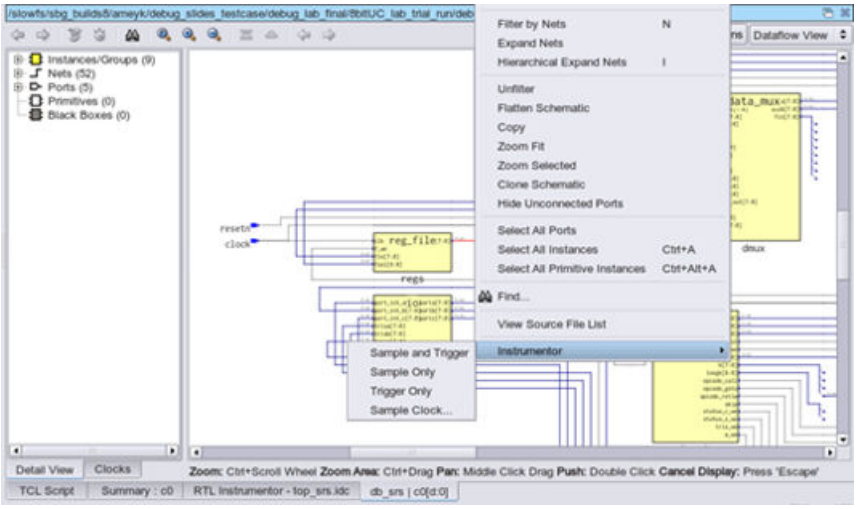
   You can now create and add a new IDC file to the design.

2. To create a new IDC file with SRS (post-compile) instrumentation, follow these steps:

   - In the pre-map stage, create a new IDC file with this command:

     ```
     edit idc -mode gui SRS_based_file.idc
     ```

   - In the GUI that opens, select the signals and set sample and trigger instrumentation for them.



3. Follow these guidelines when editing the IDC information in the file

   - Make sure the hierarchical signal path starts with the top-level module.
   - Use a period (.) not a slash (/), as the path hierarchy separator.
   - Specify signals in the generate scope with square brackets [] instead of brackets (), because there is no need to specify the range for a signal in IDC.

Feedback

Example:

```
iice clock -iice {IICE_UC} -edge positive {top.clock_c}
signals add -iice {IICE_UC} -silent -trigger -sample {top.i1.i2.s1}
```

4. Specify the post-compile instrumentation file at the next stage of the design, with the *run pre_partition* or *run pre_map* commands.

Using SystemVerilog Assertions

You can use SystemVerilog assertions (SVAs) to create an unlimited number of observation points anywhere in the design. The details of using SVAs are discussed here:

- Adding SVAs for Instrumentation
- Detecting Assertion Failures with a HAPS System Reset
- Debugging Designs with SVAs

Adding SVAs for Instrumentation

To use SVAs to define instrumentation signals, specify the assertions in the RTL. The SVAs are processed as part of RTL compilation and the compiler creates the required assertion logic. This information is translated into hardware debug signals, and the signal data is read back at runtime. See Instrumenting a Design with Unified Compile for an overview of the flow.

The following procedure describes the steps to define probe signals using SVAs. For information about defining probe signals is to use *$dumpvars*, see Using $dumpvars.

1. Do the following in the RTL source files:
   - Define the signals you want to probe by adding the SVA task to the RTL for the module.
   - Include SVAs in the RTL. You can use immediate (*assert*) or concurrent (*assert property*) SVA statements. For an example, see Examples: SVAs in the RTL.
   - If the inputs for SVAs in a module are only top ports or do not have top I/O ports, add the */*synthesis syn_noprune=1*/* synthesis directive to prevent the module containing the SVAs from being optimized away at the *run compile* stage.
2. Set additional settings in the UTF file besides the required commands.

   The UTF file contains a pointer to the essential VCS and Simon setup commands.

- Enable assertions by adding this option in the UTF file:

  ```
  assertion_synthesis -enable ALL
  ```

  Refer to the VCS documentation for a complete description of the syntax.

- If required, specify instrumentation limits in the UTF file. For more about instrumentation limits, see Setting Instrumentation Limits for UC Designs.

- Optionally, use `wire_resolution` commands to specify how multi-driver XMR conflicts are resolved. The default is WAND.

  ```
  wire_resolution -default_xmr_conflict {WAND | WOR | XMR}
  ```

  Refer to the VCS documentation for comprehensive information about the UTF commands.

3. Start the software, and compile the design with unified compile:

   - Set the *VERDI_HOME* environment variable and specify the path to the Verdi® software.
   - Set path to VCS installation: *VCS_HOME*.
   - Set this option: *option set debug_sva 1*
   - Use the `launch uc` command to run VCS unified compile on the design.

   The tool compiles the design with the VCS compiler and automatically adds the assertion signals as instrumentation signals. This is a typical command sequence:

   ```
   database load uc_debug -autocreate -technology <technology>
   option set debug_sva 1
   launch uc -utf myUTF1.utf -ucdb myUcdbOutput
   ```

4. Run through the instrumentation and debug flow described in the *HAPS Prototyping Debug Guide*, and use the Verdi interface to view and analyze the probe signals.

Grouping SystemVerilog Assertions for Instrumentation

Multiple SVA nets can be grouped together to generate a single-bit net using the `trigger_group` command in the IDC file. If one of the SVAs fail, this trigger group net will show a HIGH value in the debug waveform.

For more information, see trigger_group. For every trigger group added in the IDC file, you can see a corresponding trigger signal during runtime and in the waveform. Also, for each FPGA there will be one corresponding trigger group signal based on assertions grouped per FPGA.

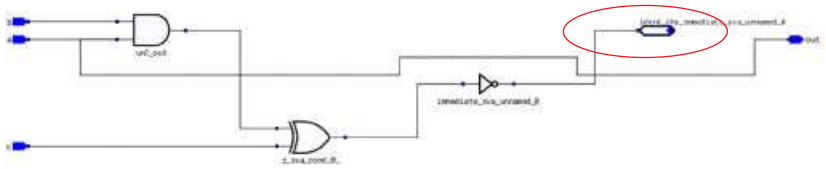
Examples: SVAs in the RTL

Feedback

You can directly add assertions to the RTL as shown below, with *p* being any property:
S1:assert property (p)

You can also add immediate or concurrent assertions. The following example shows how an immediate assertion is implemented in the design, with the schematic showing the assertion net:

```
module test (input a, input b, input c, output out);
assign out = a;
always_comb
 begin
assert ((a & b) != c);
 end
endmodule
```



Example: Command Sequence for the SVA Flow

The following snippet shows a typical sequence of commands to implement SVAs in a multi-FPGA design. The synthesis scripts for individual FPGAs include the run commands for compile, pre-map (with IDC), map, and place and route.

```
# Set SVA option
option set debug_sva 1
# Enable option in UTF file
assertion_synthesis -enable all
# Start unified compile flow
launch uc -utf run.utf -ucdb ucdb -v 2.0
# Run the flow up to System Generate
,..
# Synthesize and place and route individual FPGAs in parallel
launch protocompiler -parallel 4 -tclcmd {set ENABLE_PAR
1} -script ./db/dumpvar_sg_exp/fb1_uA/fb1_uA_srs.tcl
-script ./db/dumpvar_sg_exp/fb1_uB/fb1_uB_srs.tcl
# Export files for runtime
export runtime -path ./multi_fpga_debug
```

Detecting Assertion Failures with a HAPS System Reset

Before debugging, check for assertion failures by doing a HAPS system reset. Earlier in the cycle, you can use reports and log files to check that the assertions were implemented correctly (see Adding SVAs for Instrumentation).

The following procedure describes how to use the system reset method:

1. Configure the HAPS setup using hRun and enable a reset using the *setup control* command.
2. Configure the IICE in runtime and run it.
   - This is an example of an IICE configuration command:

     ```
     watch enable -language verilog \
     syn_identify_core.IICE_UC_INST.TriggerGroups_regular.triggrp_trig
    ger 1'b0 1'b1.
     ```

   - Click **Run**. The runtime software waits for the trigger.
3. In hRun, release the reset.

   For example, modify the reset value to 1. The runtime software detects the trigger if SVA assertion fails, and starts to download the samples. In the waveform, the trigger position appears one cycle later after the assertion failure. In case of assertion failure, the user must debug the failing condition and fix the code accordingly

   For every trigger group added in the IDC file, you can see a corresponding trigger signal in runtime.

Debugging Designs with SVAs

The tool lists all the synthesized assertions with SVA instance names that were passed through the IDC file, along with other debug signals.

1. Enable the debug signal of the SVA instance using regular runtime commands to monitor assertion failures. For example:

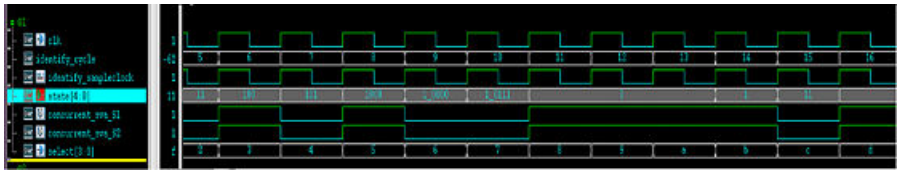
   ```
   watch enable -language verilog {top_dut.dut1.concurrent_sva_S2} {1'b0}
   {1'b1}
   ```

2. Check for signal value changes in the waveform.

   When the assertion fails, the corresponding debug signal value changes from 0 to 1 in the waveform. For details, refer to the *HAPS ProtoCompiler User Guide*.

   If you evaluate the SVA property with respect to the negedge clock when the IICE sampling is at the posedge clock, or vice versa, the SVA samples appear one cycle earlier in the waveform. For example: `assert property (@(negedge clk) req##1 grant).`

Feedback



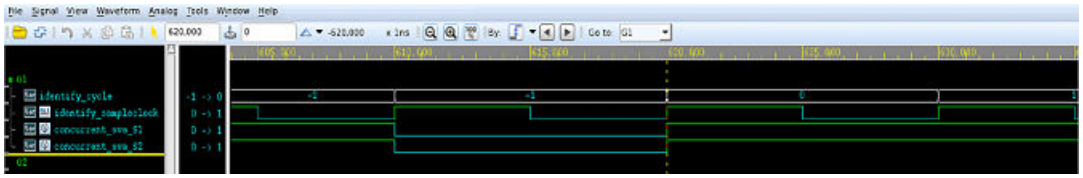
3. To filter SVAs in the waveform, use these commands:

   ◦ `signals set_property -name show_in_waveform -value 0 [hierarchy find -all -maxdepth 100 -name "*" -stat "*" -type "signal" .]`

   ◦ `signals set_property -name show_in_waveform -value 1 [hierarchy find -all -maxdepth 100 -name "*" -stat "*" -type "assert" .]`

   ◦ `write fsdb`

   The following figure illustrates the filtered SVA signals.



Using $dumpvars

SystemVerilog includes a native *$dumpvars* task construct, which provides a way to write out signal values. This provides an alternative way to instrument the design, instead of using SystemVerilog assertions (SVA). The SVA method is described in Using SystemVerilog Assertions.

- Adding $dumpvars for Instrumentation
- Using Wildcard Characters in $dumpvars
- Specifying $dumpvars for RTL Hierarchies

Feedback

Adding $dumpvars for Instrumentation

The SystemVerilog *$dumpvars* task provides a way to add RTL instrumentation to your design. Follow these steps to use the $dumpvars construct to instrument signals that you can later analyze at the debug stage:

1. Define the signals you want to probe by adding the SystemVerilog *$dumpvars* task to the RTL for the module.

   Note:

   Ensure that the *$dumpvars* task is not added inside a submodule on a signal. It is recommended to place the *$dumpvars* specification in the top module.

   If *$dumpvars* must be used within a submodule on a signal, note the following:

   - Signals cannot be referenced using the module name.
   - Signals must be referenced using the instance-based hierarchy.

   Example

   If a submodule *sub* is instantiated as *sub_inst_1*:

   $dumpvars(1, sub.signalA); is not supported.

   $dumpvars(1, sub_inst1.signalA); is supported.

- Use the *$dumpvars* task to list the variables to dump into a file specified by *$dumpfile*. You can invoke the *$dumpvars* task multiple times throughout the model. For example, in different blocks. But the tool executes all the *$dumpvars* tasks at the same time.
- Specify the signals as described in Specifying $dumpvars for RTL Hierarchies. Provide the path to the signals, instance name, or top module with the `$dumpvars` command in the RTL. All *$dumpvars* signals are instrumented as sample and trigger by default.
- Use wildcard characters (*,?) to match hierarchy or instance signal names, specified within *$dumpvars*. See Using Wildcard Characters in $dumpvars for details.

Feedback

```
initial
//Add appropriate label for grouping the following signals
begin: IICE_A
   $dumpvars (1,eight_bit_uc.uc_alu);
   $dumpvars (1,eight_bit_uc.uc_alu.U1);
end
initial
//Add appropriate label for grouping the following signals
begin: IICE_B
    $dumpvars(1, eight_bit_uc.portc);
    $dumpvars(1, eight_bit_uc.resetn);
    $dumpvars(1, eight_bit_uc.decode.skip);
end
```

2. Start the synthesis software, and compile the design with unified compile:
   - Set the *VERDI_HOME* environment variable and specify the path to the Verdi® software.
   - Set this option: *option set debug_dumpvars 1*
   - Optionally, specify instrumentation limits in the UTF file as described in Setting Instrumentation Limits for UC Designs.
   - Use the `launch uc` command to run VCS unified compile on the design. The tool compiles the design with the VCS compiler.

   For example:

   ```
   database load uc_debug -autocreate -technology <technology>
   option set debug_dumpvars 1
   launch uc -utf myUTF1.utf -ucdb myUcdb
   ```

3. Set up the debug IP.
   - Use the `edit idc` command to edit the IDC file and add the appropriate IICE settings and clock instrumentation. You can also use the GUI:



   - Set up the debug IP (IICE) and define clocks. Define required IICE settings like depth, sampling clock, sampling clock edge in the file. Define the memory buffer as *FPGA Memory* and specify a DTD sample clock.

Feedback

- Define other functionality as needed with the *iice new* and *trigger group* commands in the IDC file. This is an example of a user IDC:

```
hierarchy delimiter .
device jtagport builtin
device umr_pipe 1
iice new {IICE_UC} -type regular
iice controller -iice {IICE_UC} none
iice sampler -iice {IICE_UC} -depth 1000
iice sampler -iice {IICE_UC} -pipe 3
iice clock -iice {IICE_UC} -edge positive {top.inst1.clock}
```

4. Instrument the design.

Using Wildcard Characters in $dumpvars

Use wildcard characters (*,?) to match hierarchy or instance signal names, specified within *$dumpvars*.

- Use asterisk (*) character to match multiple characters
- Use question mark (?) to match a single character

Hierarchy or instance signal names are matched either fully or partially with wildcard characters. By using wildcard characters appropriately, you can instrument a group of signals using a single *$dumpvars* signal. Use wildcard characters with instance names within a particular scope instead of hierarchy signals.

The following example shows how the wildcard characters are used within *$dumpvars*:

```
initial
//Usage of wildcard character ? to match,
//test.u1.inst_u1.out1
//test.u1.inst_u1.out2
begin: label_1
$dumpvars(1, "test.u1.inst_u1.out?");
end

initial
//Usage of wildcard character * to match,
//test.u1.inst_u1.out1
//test.u2.inst_u1.out1
begin: label_2
$dumpvars(1, "test.*.inst_u1.out1");
end

initial
//Usage of wildcard characters ? and * to match,
//test.u1.inst_u1.out1 //test.u1.inst_u1.out2
//test.u2.inst_u1.out1 //test.u2.inst_u1.out2
begin: label_3
```

Feedback

```
$dumpvars(1, "test.*.inst_u1.out?");
end
```

Specifying $dumpvars for RTL Hierarchies

To instrument the signals, specify the `$dumpvars` command in the RTL. Provide the path to the signals, instance name, or top module. All *$dumpvars* signals are instrumented as sample and trigger by default.

Do not use *$dumpvars* to specify VHDL hierarchies. *$dumpvars* is a SystemVerilog construct and cannot be used for VHDL hierarchies. Use *$dumpport* instead for VHDL hierarchies.

1. Specify the correct paths.

   Incorrect paths result in the following error during VCS compile:

   ```
   Error-[XMRE] Cross-module reference resolution error
   ```

2. To specify complete hierarchies for *$dumpvars*, the signal hierarchies must always start with the top module name.

   Do not use *$dumpvars* (0, top) to dump the signals of all levels of the hierarchies. This syntax only dumps the signals at the top-level module. It ignores all other instances and hierarchies. It is equivalent to specifying *$dumpvars* (1, top). Dumping signals for all levels of the hierarchies is not effective. The resulting instrumentation for a large design could burden the limited hardware debug resources.

3. To dump a specific hierarchy, specify the complete hierarchical path using *$dumpvars.*

   For example:

   ```
   initial
   begin
   $dumpvars (0, top.sub1_inst.in1,
   top.sub2_inst.out1,top.sub3_inst.in3);
   end
   ```

   Although the following command is specified at the top level, it only affects the signals of the specified instance.

   ```
   $dumpvars(1, top.sub1_inst);
   ```

4. To dump signals for the current module level or submodules, use syntax like the following:

   ```
   $dumpvars(1, top.sub1_inst);
   ```

Feedback

In the following example, the first argument is a 1. So the tool dumps all variables within the module *top*. It does not dump variables in any of the modules instantiated by module top.

```
$dumpvars (1, top);
```

Similarly, the next example is specified within the *sub1_inst* module, and dumps all variables within the module sub1_inst.

```
$dumpvars (1, sub1_inst);
```

5. To dump signals for multiple modules as well as specific signals, use syntax like the following:

   ```
   $dumpvars (0, top.mod1, top.mod2.net1);
   ```

6. Do the following to set instrumentation limits:
   - To set a global instrumentation limit, use the UTF file.
   - To set an instrumentation limit on an individual module, add it to the RTL for that module with the **maxbits* attribute.

     For details about global and module-level instrumentation limits, see Setting Instrumentation Limits for UC Designs.

7. To specify instrumentation other than the default of sample and trigger, update the IDC file and run incremental debug. See Configuring an IDC File.

   You can add and delete instrumented signals or change the sample depth or trigger type.

4

Setting up the IDC and IICE

The Intelligent In-Circuit Emulator (IICE™) is an important part of the instrumentor. See these topics for information about setting up IICE units.

- Configuring an IDC File
- Configuring IICE Units

Configuring an IDC File

The Instrumentor Debugger Constraints (IDC) file defines the configuration for the hardware debug IP, along with clock specifications. These parameters are required. The IDC file can also be used to modify some parameters for incremental debug.

You can create an IDC file from the GUI or the command line.

1. To create an IDC file through the GUI, click the **Edit IICE** icon () in the instrumentor GUI.

   The dialog box that opens has multiple tabs, where you can specify various options. The most important settings are on the **IICE Sampler** and **IICE Controller** tabs.

2. To create an IDC file from the command line, use the `edit iice` command.

   The equivalent commands for the **IICE Controller** and **IICE Sampler** tabs are `iice controller` and `iice sampler`.

3. You can have multiple IICE files in a design.

4. Specify the IICE file with the run command for that stage of design.

   For example, specify it with the `run pre_map` commands.

For additional configuration, see:

- Modifying the IDC File

Modifying the IDC File

Signals can be instrumented directly within the HDL Analyst (`srs` file) outside of the instrumentor. This is a post-compile operation, and can be used in the UC flow.

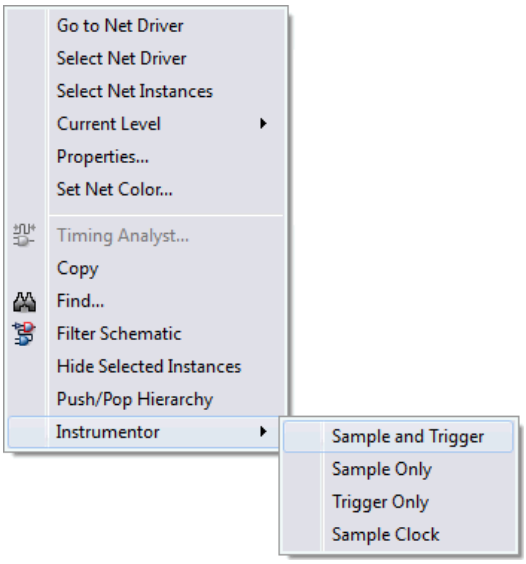
By working in the *idc* file, you can easily edit previous instrumentation. You can also use this technique to allow signals within a parameterized module. Follow these steps:

1. From a compiled state, open the schematic view by clicking the **View the schematic** icon or by typing the `view schematic` command.

   The compiled state can be from the UC flow.

2. Select the signal and specify the kind of instrumentation.

   - In the schematic view, highlight the net of the signal to be instrumented.
   - Right-click the net, select **Instrumentor** from the popup menu, and select the type of instrumentation to be applied.



   The instrumentation is automatically applied to the open instrumentor view.

3. Save the updated instrumentation and rerun compilation.

   When you open the debugger, an SRS entry is included in the hierarchy browser. Selecting this entry displays the additional signals added to the instrumentation.

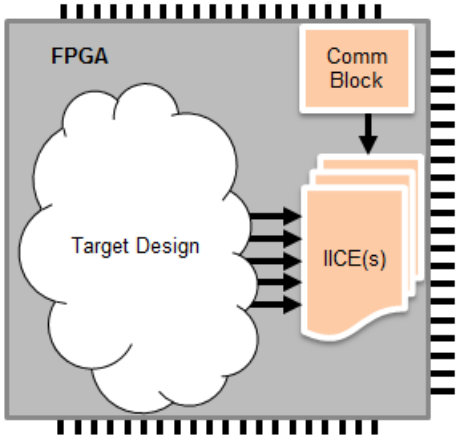
   Note that trigger expressions on signals added to the instrumentation must use the VHDL style format.

Configuring IICE Units

Intelligent In-Circuit Emulator (IICE™) units are blocks for probe and communication logic to trigger and sample the design.

IICE consists of logic added to the design for proper communication with the debugger. It acts as an on-chip logic analyzer, and contains its own sample buffer and trigger logic. Each IICE can be configured and debugged independently. One IICE can trigger another.

Figure 2 IICE Unit



An IICE unit can be auto-inserted or manually configured:

- ProtoCompiler automatically inserts a debug IP for preparing the design for instrumentation. Users can prepare the design for debug and use Always On Debug (AOD) IICE for instrumenting the design. For more information, see Always On Debug IICE.
- Using the Instrumentor, you can manually define one or more IICE units per the design. Multiple IICE units allow signals to be triggered and sampled from different clock domains within a design. Each IICE unit can have unique IICE parameter settings including sample depth, sampling/triggering options, and sample clock and clock edge. During the debugging phase, individual or multiple IICE units can be armed. For more information, see Configuring Individual IICE Parameters.

  Parameters that are common to all IICE units are set in the **Instrumentor Preferences** dialog box. See Configuring Instrumentor Preferences.

The following topics describe Always On Debug (AOD) IICE and manual configuration of IICE units.

- Configuring IICE Units
- Configuring Individual IICE Parameters

Feedback

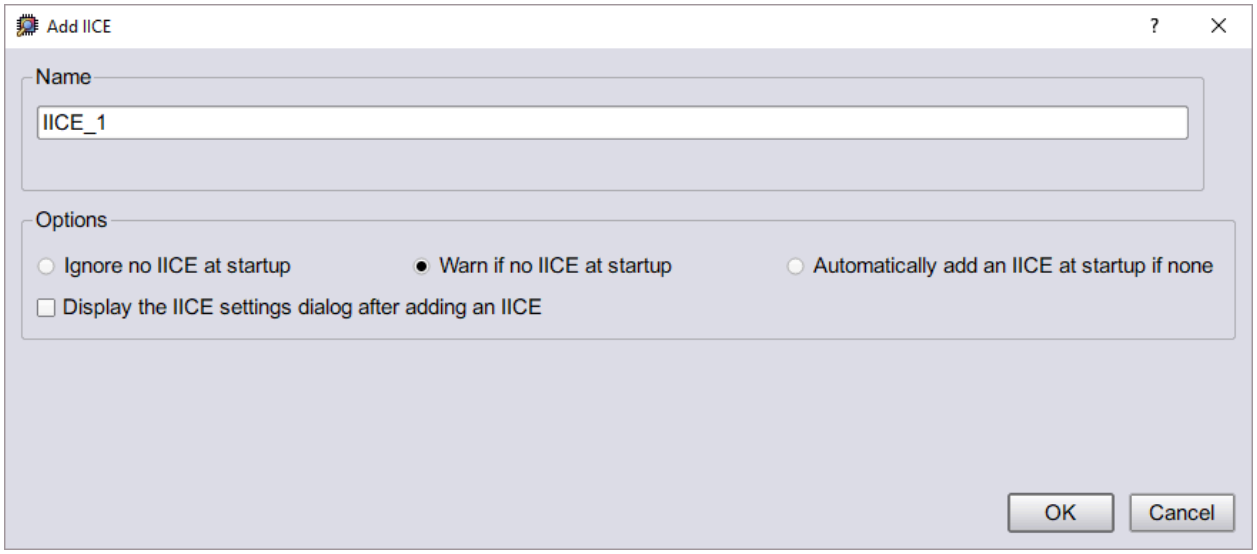
- Configuring IICE Sampler Parameters for an IICE
- Configuring IICE Sample Clock
- Configuring IICE Controller and Other Options
- Configuring Multiple IICE Units

Configuring IICE Units

You can add an IICE from the instrumentor GUI or from the command line. The IICE information is stored in the IDC file.

1. To add an IICE from the instrumentor GUI, click the **Add IICE** icon ( ) or select the **Instrumentor > IICE > Add IICE**.

   The **Add IICE dialog** box opens to allow you to define the type and name of the new IICE unit.



   - You can add multiple IICE units to a design.
   - The recommendation is to create a different IICE for each distinct clock domain.

2. To add an IICE unit with a command, add the `iice new` command to the IDC file.

```
iice new [iiceID] -type regular
```

3. Specify parameters for the IICE:
   - Type a name for the IICE. The default IICE name is formed by adding an integer suffix to the word *IICE* (for example, *IICE 0*, *IICE 1*).
   - Select **Regular** (the default) to add a normal IICE unit.
   - Set parameters that are shared by all IICE units as described in Configuring Instrumentor Preferences.
   - Set parameters that are unique to this IICE. Make sure to define the sample clock for the IICE. See Configuring Individual IICE Parameters.

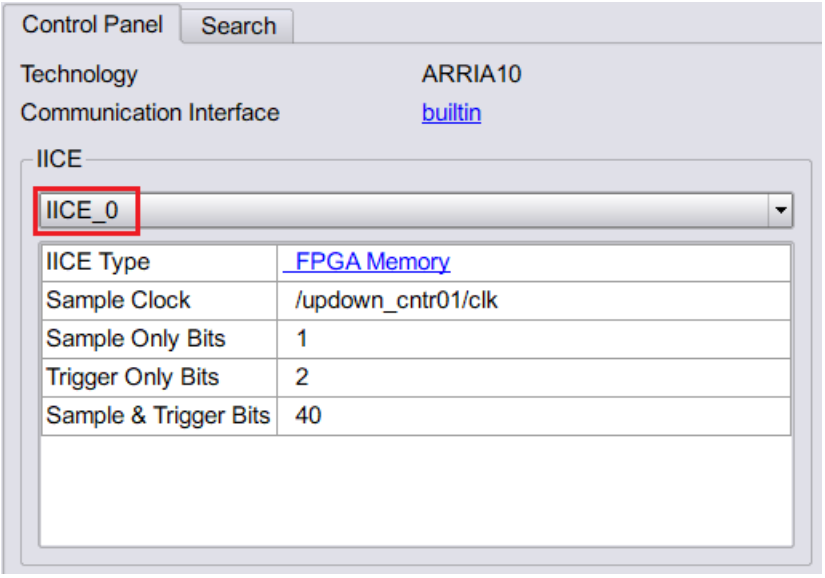
   The IICE information is stored in the IDC file.
4. To modify information for an IICE, use one of these methods:
   - From the GUI, select the IICE and click the **Edit IICE** icon () or select **Instrumentor > IICE > Edit IICE**..
   - Edit the IDC file directly, using the `edit idc file.idc` command.
5. Follow these steps to delete an IICE unit:
   - If there is more than one IICE unit, first select the IICE to delete in the **Control Panel** tab.
   - To delete the IICE, choose **Instrumentor > IICE > Delete IICE** from the menu bar.

Configuring Individual IICE Parameters

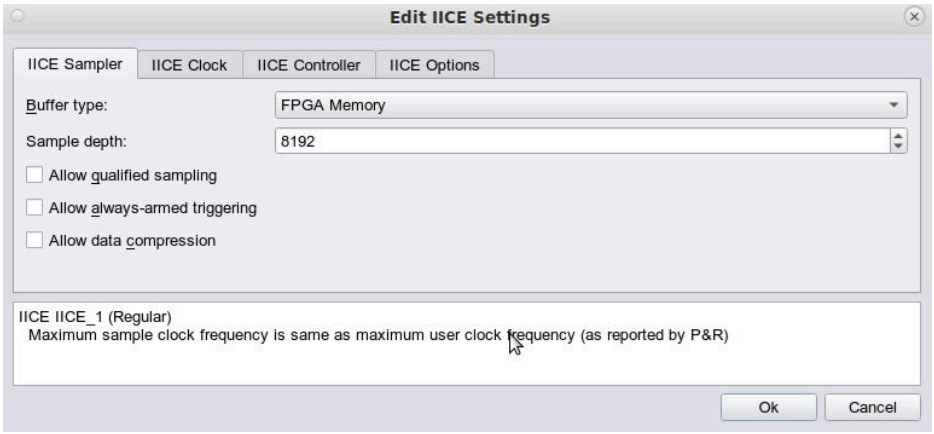
Follow these instructions to set individual IICE parameters. For information about setting parameters that are common to all IICE units, see Configuring Instrumentor Preferences.

1. If there are multiple IICE units, first select the IICE.

   Go to the **Control Panel** and select the IICE to work on.



2. Open the **Edit IICE Settings** dialog box for individual parameters using one of these methods.
   - In **Control Panel**, click the entry for **IICE Type** field.
   - Click the **Edit IICE** icon ( ) to reset or change parameters.



   - Edit the IDC file directly with the `edit iice` command.

   Set individual IICE parameters from this interface, with the main tabs being **IICE Sampler**, **IICE Clock**, and **IICE Controller**. The equivalent commands in the IDC file are `iice sampler`, `iice clock`, and `iice controller`.

3. On the **IICE Sampler** tab, specify parameters for the memory buffer to be used for debugging.

Feedback

See Configuring IICE Sampler Parameters for an IICE.

4. On the **IICE Clock** tab, specify the sample clock to be used for the IICE.

   See Configuring IICE Sample Clock.

5. Set trigger options for the IICE on the **IICE Controller** tab.

   See Configuring IICE Controller and Other Options.

Configuring IICE Sampler Parameters for an IICE

The **IICE Sampler** tab is used to define parameters for the memory to be used for sample storage during debug. You must define buffer parameters for each IICE.

1. If there are multiple IICE units, first select the IICE.

   Go to **Control Panel** and select the IICE to work on.

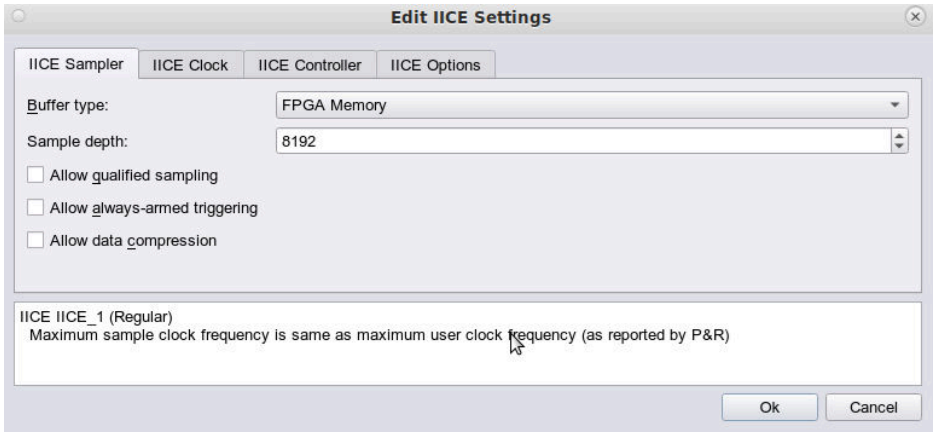
2. Click the **Edit IICE** icon ( ), open the **Edit IICE Settings** dialog box, and go to the **IICE Sampler** tab.

   The command equivalent is `edit iice`

   - To use the command line, include this command in the *idc* file:

     ```
     iice sampler -iice {iiceID| all} bufferType
     ```

   .



3. Configure the IICE sampling parameters.

The **IICE Sampler** tab is the default tab and defines buffer parameters and sampling options.

Buffer Type

The **Buffer type** parameter specifies the type of RAM to be used to capture the on-chip signal data. The default is *FPGA Memory*. The haps_DTD_cross_trigger setting configures the IICE to use the DDR3 memory connected to each FPGA (partition flow).

Sample Depth

The **Sample depth** parameter specifies the amount of data captured for each sampled signal. Sample depth is limited by the capacity of the FPGAs implementing the design. But it must be at least 8 due to the pipelined architecture of the IICE.

If the block RAM used in the design is minimal, more signal data can be captured in the block RAM. If most of the block RAM is used for the design, only limited RAM is available for signal data. In this case, use logic RAM. The sample depth increases significantly with the deep-trace debug feature.

The appropriate value is determined by the type of debug you are planning to run. Deep trace debug (DTD) schemes could require more buffer memory. Also check the RAM usage to determine signal depth. If the design only uses some FPGA RAM resources, you can use RAM to capture signal data.

Allow Qualified Sampling

Enable **Allow qualified sampling** to build an IICE block that samples a single time step of data for selective viewing. One data value is sampled each time the trigger condition is true. With qualified sampling, you can follow the operation of the design over a longer period of time. For example, you can observe the addresses in a number of bus cycles. Sample only one value for each bus cycle instead of a full trace. Using qualified sampling includes a minimal area and clock-speed penalty.

Allow Always-Armed Triggering

When enabled, **Allow always-armed triggering** saves the sample buffer for the most recent trigger until the next trigger or until it is interrupted. With this option, the tool takes a snapshot each time the trigger condition becomes true.

With always-armed triggering, you always acquire the data associated with the last trigger condition prior to the interruption. This mode is helpful to analyze a design that uses a repeated pattern as a trigger and then randomly freezes. For example, bus cycles. You can retrieve the data corresponding to the last time the repeated pattern occurred prior to freezing.

Allow Data Compression

When enabled, **Allow data compression** adds logic to the IICE to compress stable sample data in the debugger. When unchecked (the default), compression logic is excluded from the IICE, and data compression in the debugger is unavailable. Note that

Feedback

there is a logic data overhead associated with data compression. Do not check this box when sample data compression is not used during debug.

Configuring IICE Sample Clock

You must define a sample clock for each IICE so that the tagged points can be used. It determines when signal data is captured by the IICE. In addition, you can use oversampling and define multiple sub-sample clocks for the same IICE. You can do this through a command or from the GUI.

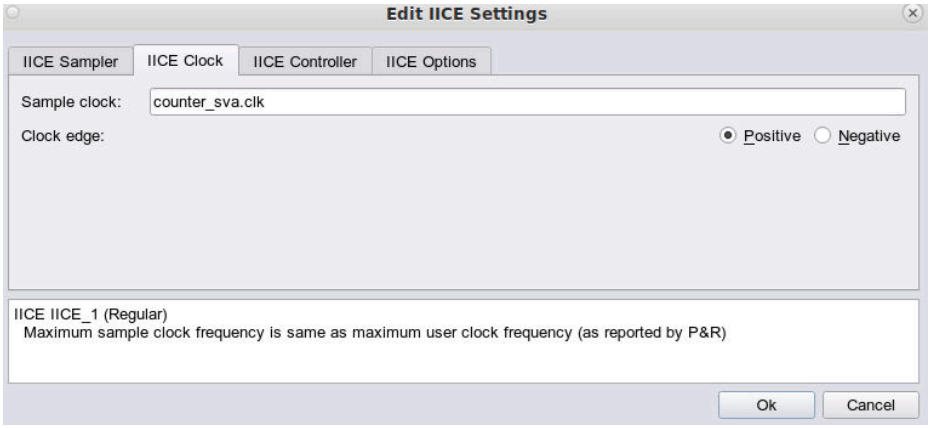
1. Determine the signal to use as the sample clock, using the following criteria:
   - The clock signal must be a single-bit scalar signal.
   - The source of the sample clock must be the output of a BUFG or IBUFG component.
   - In general, make sure sampled signals are synchronous with the sample clock or are synchronous with a multiple of the clock, so that the sampled signals are stable when the specified edge of the sampled clock occurs.
2. Mark the signal as the sample clock, using one of these methods:
   - To use the command line, specify the *iice clock* command. Specify the clock edge to be used for triggering with the *-edge* argument. See iice for the complete syntax and options.
   - From the GUI, do the following:

     If there are multiple IICE units, first select the IICE.

     Go to **Control Panel** and select the IICE to work on.

Feedback

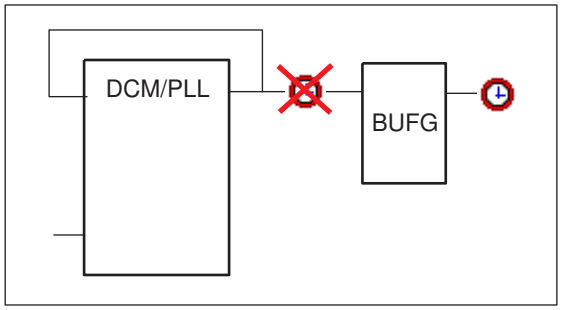
Click the **Edit IICE** icon ( ), open the **Edit IICE Settings** dialog box, and go to the **IICE Clock** tab. Specify the sample clock and the clock edge to be used for triggering, and click **OK**.



3. Specify the complete hierarchical path of the signal in the **Sample clock** field.

   The **Sample clock** parameter determines when signal data is captured by the IICE. The sample clock can be any signal in the design that is a single-bit scalar type. Enter the complete hierarchical path of the signal as the parameter value.

   Select the sample clock carefully. Signals are sampled on the edge of the clock. For valid sample values, the signals must be stable when the specified edge of the sample clock occurs. Usually, the sample clock is the same clock that the sampled signals are synchronous with. The source of the sample clock must be the output from a BUFG/IBUFG device.



   - Refer to the guidelines (Guidelines for Selecting a Sample Clock) to select a valid sample clock. Usually, the sample clock is either the clock that the sampled signals are synchronous with or a multiple of that clock.

4. The **Clock edge** options determine if samples are taken on the **Positive** or **Negative** edge of the sample clock. The default is the positive edge.

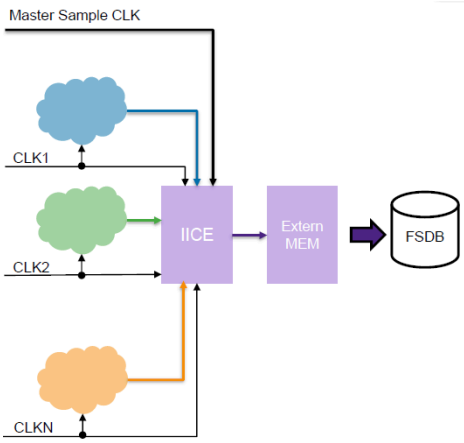
Feedback

Specify a positive or negative clock edge for sampling.

Signals are sampled on an edge of the clock. For the sample values to be valid, the signals being sampled must be stable when the specified edge of the clock occurs. So they must be synchronous with the clock or a multiple of the clock.

Defining Multiple Sub-Sample Clocks for Oversampling

With multi-clock support in one IICE, signals within an IICE can be triggered internally, so cross-triggering is not required. This eliminates the need to use different IICEs for asynchronous clocks. Instead, oversample different clocks by adding them to clock groups and synchronizing these clock domains to an oversampling clock domain. Both IICE and memory operate on the sample clock domain. There is one debug controller per design and a single waveform view.



1. Specify the sample clock.

   Use the *iice clock* command or one of the other methods described in Guidelines for Selecting a Sample Clock. The sample clock is used as the fast sample clock for oversampling. The sample clock frequency must be at least as fast as the fastest user-defined sample clock domain.

2. In the IICE file, specify the clock signal for the user-defined clock domain with the *clock_group* command.

   - These clocks must be within the same IICE.
   - The sub-sample clocks must be the same speed or lower in frequency than the fast sample clock.
   - The sub-sample clocks can be asynchronous to the fast sample clock

Feedback

- Specify the *clock_group* command:

  ```
  clock_group add -name value -freq value -edge value [signal] [-iice value]
  ```

  For command syntax details, see clock_group.

  Signals that are not defined as part of a clock group are sampled in the fast clock domain. The signals that are defined are sampled at the speed of the clocks defined for that domain.

3. When tagging signals for sampling and triggering, do the following:
   - Include the clock group name:

     ```
     signals add  [signal] [-iice value] [-sample] [-trigger] [-msb value] [-lsb value] [-field value] [-silent] [-clkgroup value]
     ```

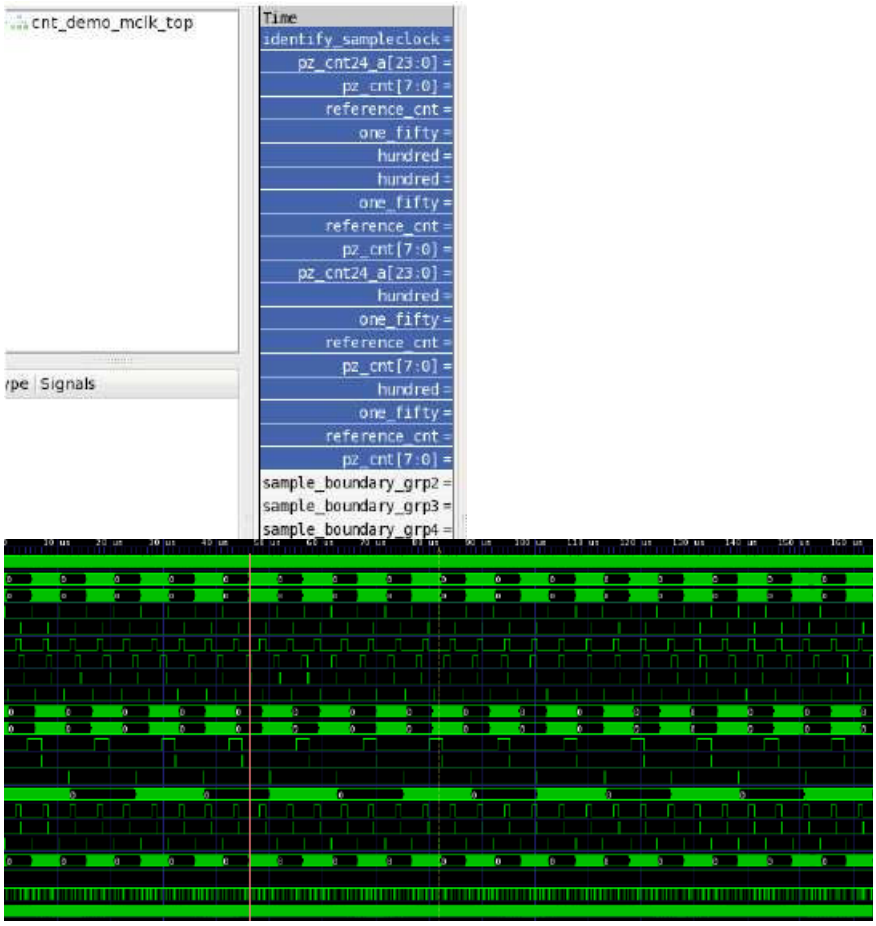
     For command syntax details, see signals.

4. View the waveform.

   The minimum resolution for the displayed waveforms is the fast sample clock period, but the sub-sample clocks could be asynchronous to the fast sample clock.

Feedback

The figure shows sample data for signals in different clock groups. The *sample_boundary** signals specify when the sample data is valid.

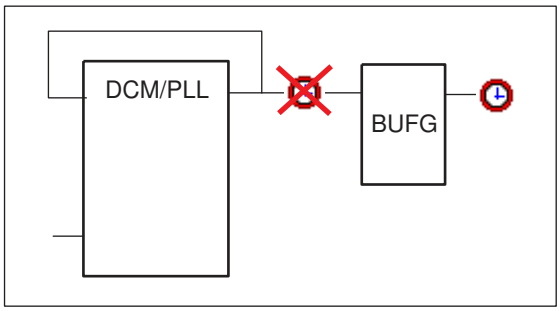


Guidelines for Selecting a Sample Clock

Follow these guidelines when selecting a sample clock for an IICE:

- The sample clock must be a single-bit scalar signal.
  - For the UC flow with *$dumpvars* instrumentation, the sample clock must be a top-level clock (top.clk).
  - For post-compile SRS instrumentation, the sample clock must be a signal from the compiled database.
- Make sure that a clock buffer is assigned to the selected signal. The source of the sample clock must be the output from a BUFG/IBUFG device.

Feedback



- Signals are sampled with respect to the sample clock so the clock must be synchronous or a multiple.
- The sample clock must be capable of being sampled by the instrumentor.
- If the IICE is to be cross-triggered from another IICE, the clocks for the affected IICE units must work together.

Configuring IICE Controller and Other Options

Triggering modes can be broadly classified as simple and complex. Simple mode is the default. Complex triggering modes use advanced triggering techniques that use counters or state machines. For example, state machine-based triggering sets triggers based on a complex sequence of events:

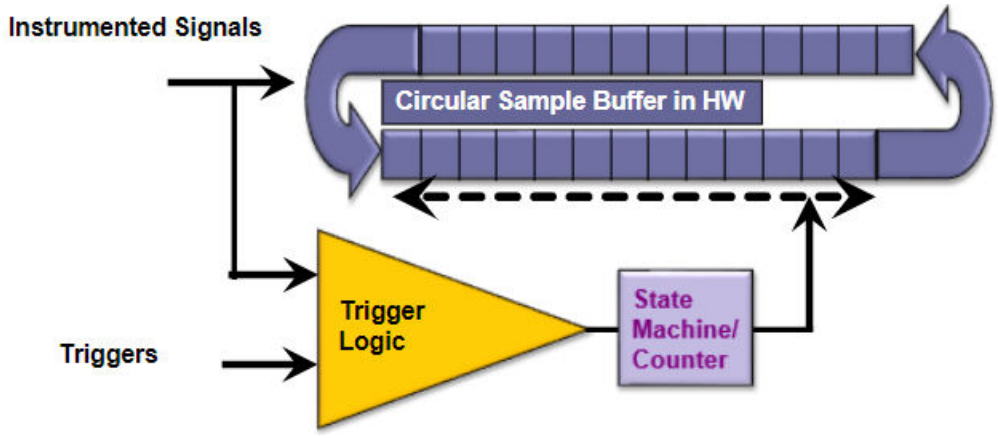
“trigger if pattern A occurs exactly five cycles after pattern B, but only if pattern C does not intervene.”

Set up the trigger through the instrumentor, and then program and activate the trigger at the debug stage.

Trigger Logic and Sample Buffer

The trigger halts sampling, but not the hardware.

Feedback



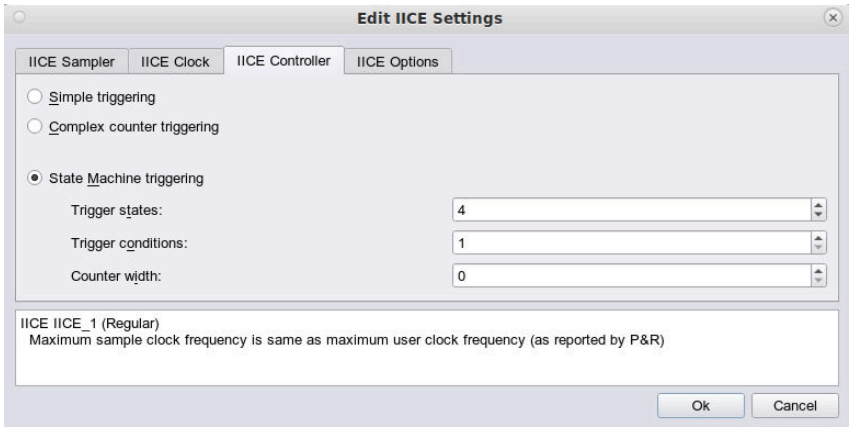
You must set triggering options for each IICE individually. All triggering options affect the area cost of the IICE.

1. If there are multiple IICE units, first select the IICE.

   Go to **Control Panel** and select the IICE to work on.

2. Click the **Edit IICE** icon (), open the **Edit IICE Settings** dialog box, and go to the **IICE Controller** tab.

   The equivalent command is `edit iice -gui`. In the IDC file, the commands are `iice controller` commands.



3. Set the triggering mode: **Simple triggering**, **Complex counter triggering**, or **State Machine triggering**.

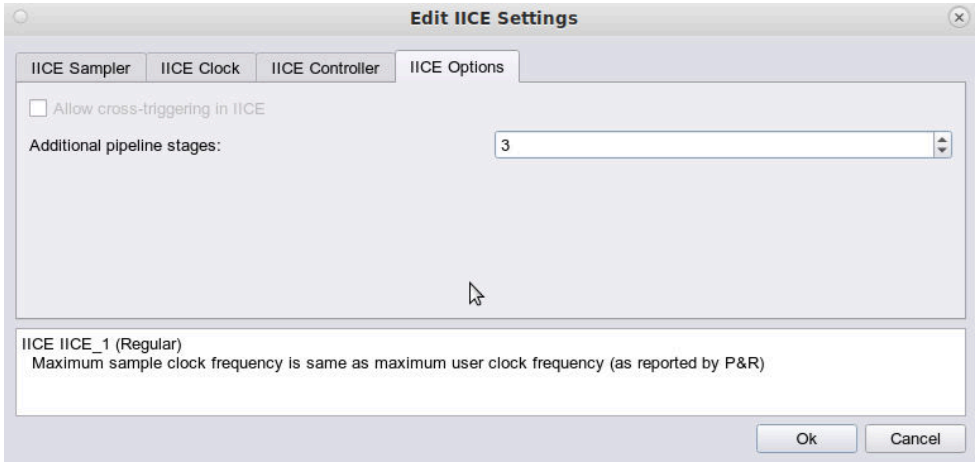
   - **Simple triggering**: See Setting up Simple Triggers.
   - **Complex-counter triggering** augments the simple triggering by instrumenting a variable-width counter that can be used to create a more complex trigger function.

Feedback

Use the width setting to control the required width of the counter. See Setting up Complex Counter Triggers.

- **State Machine triggering** allows you to pre-instrument a variable-sized state machine that can be used to specify a flexible trigger condition. Use **Trigger states** to customize how many states are available in the state machine. Use **Trigger conditions** to control how many independent trigger conditions can be defined in the state machine. For more information on state-machine triggering, see State Machine Triggering and Setting up State Machine Triggers.

4. Use the **IICE Options** tab to configure external triggering. You can import a trigger from an external source and configure it as a trigger condition for the active IICE. The external source can be external logic on the board or an IICE located on another device.



Cross-triggering allows a source external to the IICE to be the trigger.

Allow cross-triggering in IICE

Select the **Allow cross-triggering in IICE** check box to allow this IICE unit to accept a cross-trigger from another IICE unit.

Additional Pipeline Stages

The **Additional pipeline stages selection** allows additional pipeline registers to be introduced for high-speed sampling of the instrumented signals that cannot meet timing. Based on a user-specified value, an equivalent number of registers are dynamically inserted for every bit to be instrumented. The number of registers to be added is selected from the adjacent drop-down menu.

Feedback

For more information, see:

- Setting up Simple Triggers
- Setting up Complex Counter Triggers
- Setting up State Machine Triggers
- Setting up an IICE for Cross-Triggering
- Setting up RTL Cross Triggers

Setting up Simple Triggers

Simple triggering creates a trigger condition for capturing the sample data.

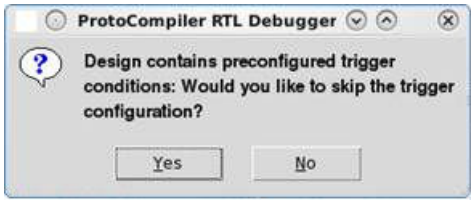
1. With simple triggering, the tool compares signals to values (including don't cares) and triggers when the signals match the values.

   All signals must match their respective comparison values in order to trigger.

   You can now enable the trigger when you debug the design.

2. To preconfigure a trigger for debugging initial cycles, add the *preconfigure* argument when you designate signals with the *signals* commands in the instrumentor.

   Preconfiguring sets specific trigger conditions in advance for the debugger, and arms the IICE to run immediately after configuration. This lets you trap error conditions that might occur during the first several cycles. When you start the debugger with preconfigured signals, a prompt asks if you want to run them. To download the sample data with the pre-configured trigger, select *Yes*. To ignore the pre-configured trigger and enable normal sampling, select *No*.

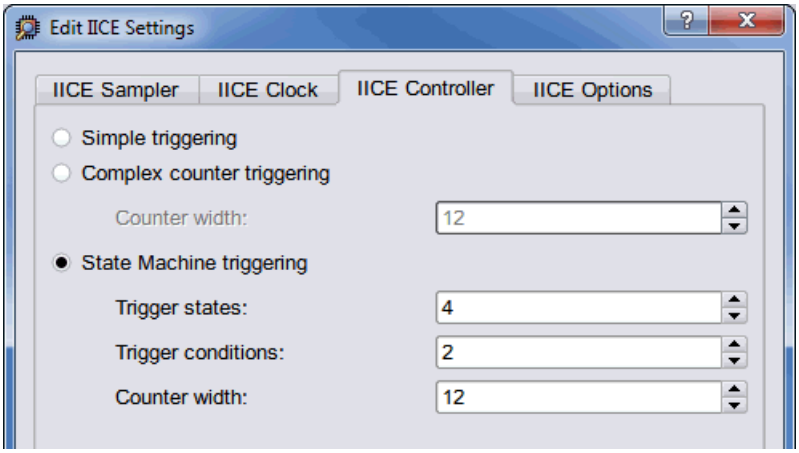


Setting up Complex Counter Triggers

Complex-counter triggers augment simple triggering by instrumenting a variable-width counter that can be used to create a more complex trigger function.

Feedback

1. Define a complex counter in the instrumentor.
   - In the instrumentor, use the `iice controller` command or go to the **IICE Controller** tab of the **Edit IICE Settings** dialog box to create the counter.



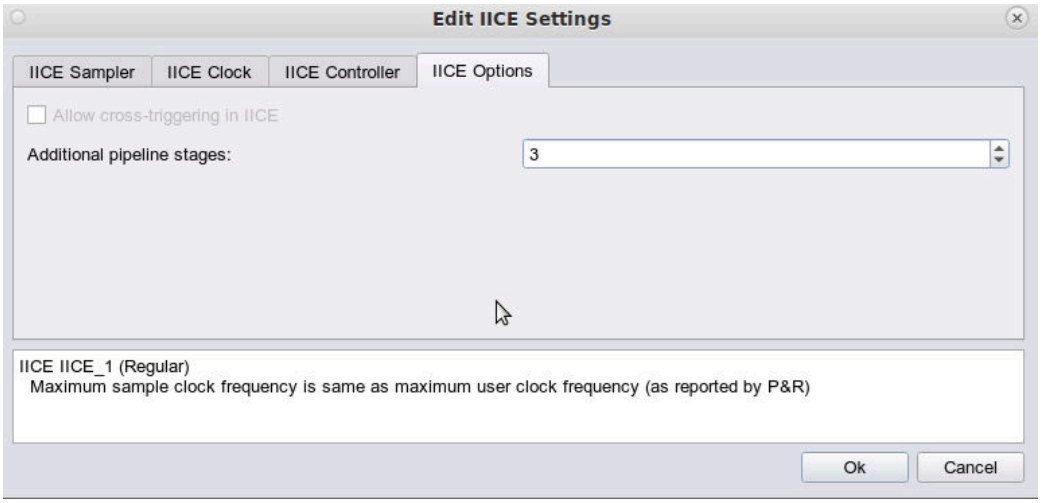
   - Enable **Complex counter triggering**.
   - Set the counter width, which determines the width for the counter. The default is *16*, which implements a 16-bit counter with a range from 0 to 65535. If you set the counter width to *0*, no counter is inserted in the design.

     The equivalent commands are shown below:

```
iice controller counter 1
iice controller counterwidth <value>
```

2. Set other options for the counter trigger.
   - To enable cross triggering, use the following instrumentor command or enable the **Allow cross-triggering in IICE** option on the **IICE Options** tab, as shown in the figure below.

```
iice controller -crosstrigger 1
```



- On the **IICE Sampler** tab, set the buffer type. Configure the **Sample depth** for the buffer with the following instrumentor command:

  ```
  iice sampler -depth 2048
  ```

- Set the sampling mode on the same tab. For example, you can enable **Allow always-armed triggering**.

3. Add the counter to the design.

   You can now activate the trigger when you debug the design.

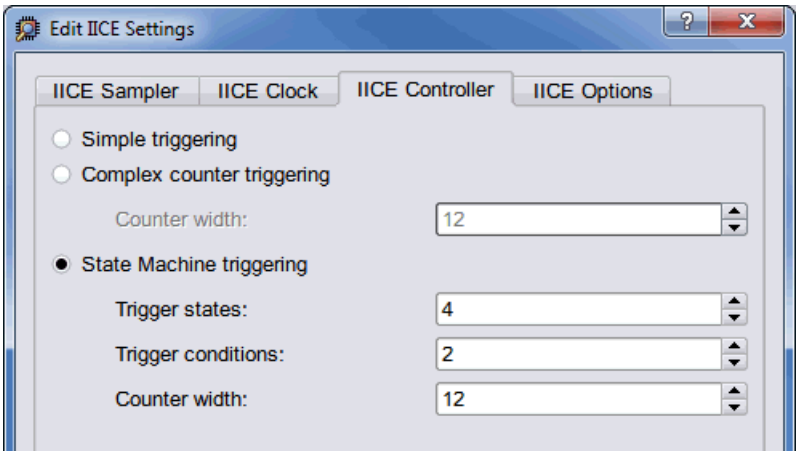
Setting up State Machine Triggers

State machine triggers are based on a variable-sized state machine that specifies a flexible trigger condition. With this method, you use sequences of trigger conditions to create counter-based state machines and create very effective triggers. A common design configuration is to trigger when a specific sequence of events occurs, which in turn causes data collection to stop and the sample data to be downloaded from the FPGA by the debugger executable.

Set up the state machine trigger during instrumentation and then program the state machine dynamically during debug, to create a complex, design-specific trigger. Trigger conditions can be set at the instrumentation stage as described here, but can also be set dynamically in the debugger.

Feedback

1. Open the dialog box.
   - Select **Instrumentor > IICE > Edit IICE** from the menu bar or click the **Edit IICE** icon.
   - In the instrumentor **Edit IICE Settings** dialog box, go to the **IICE Controller** tab.



2. Enable **State Machine triggering**.

   The command equivalent is `iice controller statemachine`.

3. Specify **Trigger states**, **Trigger conditions**, and **Counter width** in the corresponding fields.
   - Use **Trigger states** to define the number of states available in the state machine. To use a state machine for cross-triggering it must have at least three states. The range is 2 to 16; powers of 2 are preferable as other numbers limit functionality and do not provide any cost savings.

     This is the most cost-critical setting, because each trigger condition results in an additional address bit on the RAM and doubles the size of the RAM table with each bit.
   - In **Trigger condition**, specify the number of independent trigger conditions in the state machine. The range is from 1 to 16. All trigger conditions are identical in terms of signals, but they can be programmed separately in the debugger.
   - If the state machine includes a counter, use **Counter width** specify a value greater than 0 for the desired width of the counter. A counter in the state machine augments the functionality of the state machine, just as with a simple trigger. Counter width is a direct contributor to the width of the RAM table, and this might be significant for FPGA RAM primitives that allow a trade-off of width for depth.

     See State Machine Implementation for Triggering for details about the implementation.

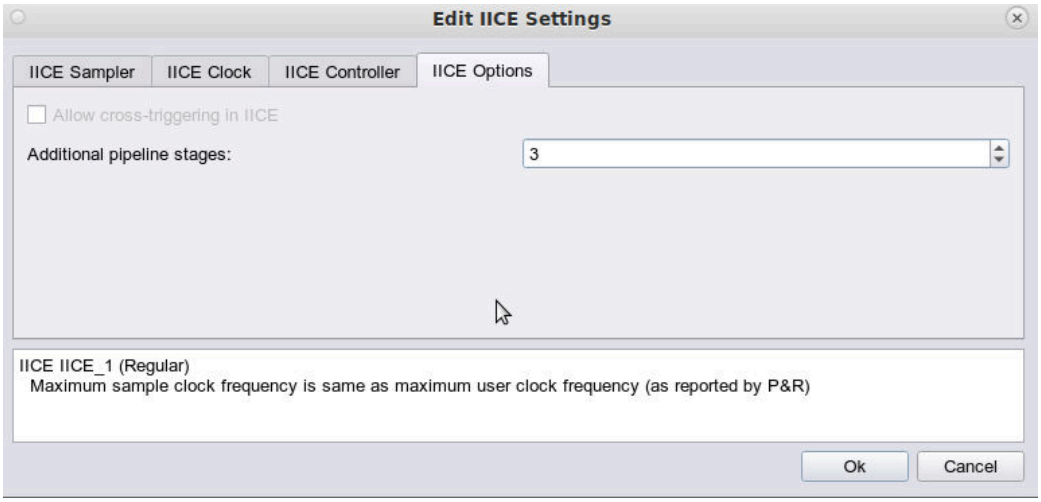
Feedback

This example shows a sequence of commands that sets up a state machine for triggering, with two trigger conditions and two trigger states.

```
iice controller -iice IICE statemachine
iice controller -iice IICE -counterwidth 4
iice controller -iice IICE -triggerconditions 2
iice controller -iice IICE -triggerstates 2
```

4. Set other options for the state machine trigger.
   - To enable cross triggering, use the following instrumentor command or enable the **Allow cross-triggering in IICE** option on the **IICE Options** tab, as shown in the figure below.

     ```
     iice controller -crosstrigger 1
     ```

   - On the **IICE Sampler** tab, set the buffer type to *internal_memory*. Configure the **Sample depth** for the buffer with the following instrumentor command:

     ```
     iice sampler -depth 2048
     ```

   - Set the sampling mode on the same tab. For example, you can enable **Allow qualified sampling** or **Allow always-armed triggering**.
5. Check the implementation to ensure that resources are being used effectively.

   For example, it may be advantageous in a particular situation to trade away states for additional trigger conditions or for increased counter width. You can make these changes at the debug stage as well.
6. Add the state machine to the design.

   You can now activate triggers when you run debug.

Feedback

State Machine Implementation for Triggering

For each trigger condition *ci*, the tool implements a logic cone that evaluates the signals connected to the trigger logic and culminates in a 1-bit result identical to the trigger condition in simple mode. All the 1-bit results are connected to the address inputs of a RAM table.

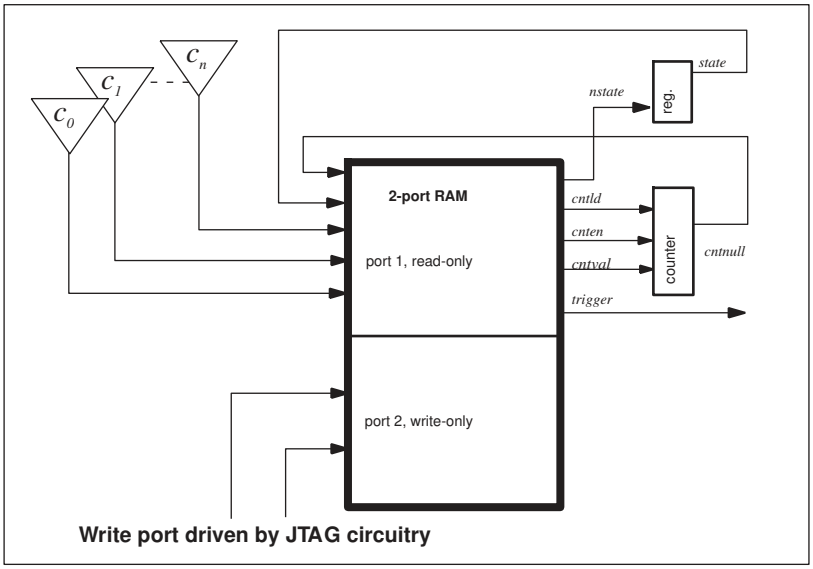
If the state machine includes a counter, the counter output is compared to constant 0, and the single-bit output of that comparison is also connected to the address inputs of the same RAM table. The other address inputs are provided by the state register.

The implementation of the RAM table is identical to the implementation of the sample buffer (that is, the *device buffertype* setting selects the implementation of both the sample buffer and the state-machine RAM table).

The outputs of the RAM table are the next-state value *nstate*, and the trigger signal *trigger* (which causes the sample buffer to take a snapshot if high).

If the state machine includes a counter, there are additional outputs:

- The counter-enable signal *cnten* (if ‘1’, the counter is decremented by 1)
- The counter-load signal *cntld* (if ‘1’, counter is loaded with *cntval*)
- The counter value *cntval* (only useful in conjunction with *cntld*)



Setting up an IICE for Cross-Triggering

Cross-triggering allows the trigger from one IICE unit to qualify a trigger on another IICE unit, even when the two IICE units are in different clock domains. The IICE to be cross-triggered is called the *destination* IICE and the IICE initiating the trigger is called the

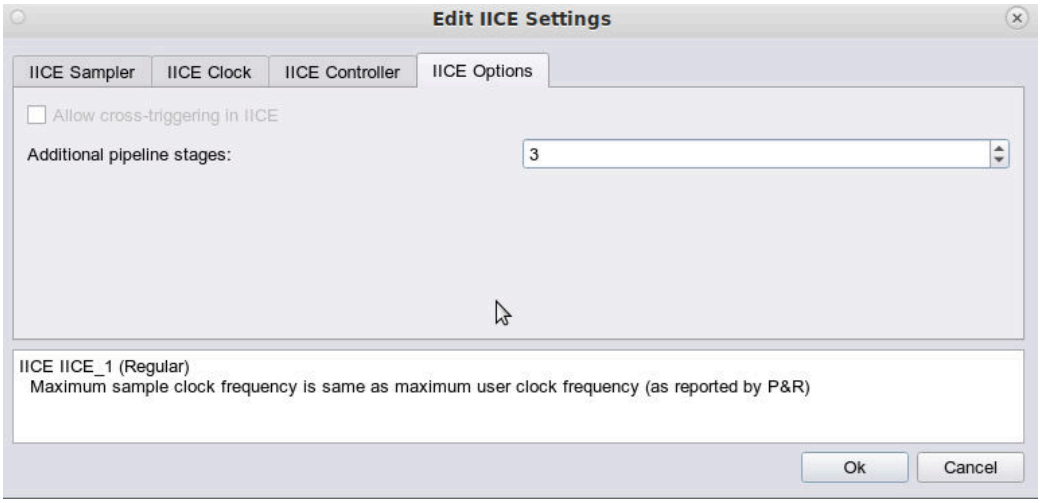
Feedback

source IICE. Multi-IICE cross-triggering provides a way for subsystems in an SOC to interact and synchronize by exchanging debug triggers. You can cross-trigger between IICE units on different FPGAs.

You can use cross-triggering with simple triggers and complex counters. State machines support cross triggering by allowing the IICE unit IDs to be included in state machine equations.

The following procedure describes how to set up an IICE for cross-triggering. This method is useful to cross-trigger IICEs that are at same level, such as system-level IICEs or IICE units in the same partitioned FPGA. For IICEs at different levels, use the method described in Setting up RTL Cross Triggers.

1. Define the IICE in the instrumentor.
2. For a *destination* IICE, enable cross-triggering in the instrumentor.
   - From the GUI, enable cross-triggering by going to the **IICE Options** tab of the **Edit IICE Settings** dialog box, and enable the **Allow cross-triggering in IICE** check box:



   This is the equivalent command:

```
iice controller -iice {IICE_0 IICE_1 ... IICE_n} -crosstrigger
```

3. Instrument the signals in each IICE and complete the Bit file generation flow
4. Download the design on the hardware using hRun.
5. Control cross-triggering.
6. Run the IICE and observe the waveforms.

Feedback

Setting up RTL Cross Triggers

This method sets up cross-triggering by instantiating IICE blocks in the RTL and then instrumenting the *trigger_out* signal of an IICE in other IICEs. The RTL cross-trigger method is well-suited to triggering IICEs that are at different levels. For example, use it cross-trigger between a system IICE and a partitioned FPGA IICE, or for cross-triggering IICEs in multiple partitioned FPGAs. For another method of cross-triggering, see Setting up an IICE for Cross-Triggering.

1. Instantiate IICE blocks in the RTL.

   Do this by adding the `<installDir>/lib/di/haps_controlled_clocks.v IICE` definition file to the source list, which is passed to the **`run compile`** command.

```
(* syn_noprune, syn_keep *) wire  trigger_o_IICE_1, trigger_o_IICE_2, trigger_o_IICE_3 ;
(* syn_noprune *) iice_trigger #("IICE_1") trigger_inst_IICE_1 (trigger_o_IICE_1);
(* syn_noprune *) iice_trigger #("IICE_2") trigger_inst_IICE_2 (trigger_o_IICE_2);
(* syn_noprune *) iice_trigger #("IICE_3") trigger_inst_IICE_3 (trigger_o_IICE_3);
```

2. In the instrumentor, instrument the trigger outputs of the IICE blocks in each other.

   The following IDC file example illustrates this:

```
iice new {IICE_1} -type regular
iice controller -iice {IICE_1} none
iice sampler -iice {IICE_1} -depth 2048
iice clock -iice {IICE_1} -edge positive {/SRS/osc_ext}
signals add -iice {IICE_1} -sample -trigger {/SRS/reset_n}\
{/SRS/u_cnt_demo_inst1/capture_flag}\
{/SRS/trigger_o_IICE_2}\
{/SRS/trigger_o_IICE_3}\
{/SRS/trigger_o_IICE_4}
```

3. Partition the design such that the IICE blocks and the trigger outputs goes into the respective FPGAs, where the instrumented signals are partitioned.
4. Generate bit files for each FPGA.
5. Export the design.
   - Use **`export runtime`** command in the ProtoCompiler tool.
   - If the IICEs are at different FPGA levels, there is separate debug project (*debug.prj*) created for each FPGA.
6. Program the hardware.
7. Run the debugger. The IICE will move to the *waiting for trigger* state as the trigger from other IICEs has not been triggered yet.

8. Open another Runtime session and open the debug project for the IICE that is to be used to crosstrigger the previous running IICE which was in *waiting for trigger* state.
9. Run the debugger tool.

   After the trigger is detected, a new debugger session is opened. The previous Debugger session also gets triggered and the waveforms are captured in it.

Guidelines

- For IDC cross-triggered IICE of BRAM type at the same level (system or SLP):
  - There is latency of five clock cycles in Runtime waveform if the IICE type is *Simple* or *Complex counter triggering*.
  - There is latency of four clock cycles in Runtime waveform if the IICE type is *State machine triggering*.
- For IDC cross-triggered IICE of DTD type, the latency is unknown because runtime is not aware of DTD IICE link delay. But you can capture the correct sample and waveforms after cross-triggering.

Configuring Multiple IICE Units

See the following topics for information about working in a multi-IICE design.

- Using Multiple IICEs for Multiple Instrumentation Modes
- Examples of $dumpvars Labels for Multi-IICE Grouping

Using Multiple IICEs for Multiple Instrumentation Modes

If you use multiple instrumentation modes in the same design, route the different probe signals to separate IICE blocks. For example, *$dumpvars* and SVA. To do this, specify different IICE blocks with appropriate debug IP settings and clock instrumentation. You can specify the blocks in a single IDC file or in individual IDC files.

1. In the IDC file, define separate IICE blocks for each instrumentation mode using the `iice new` command.

   See iice new for an overview of the syntax.

   The following example shows how the `iice new` command is used to define two separate IICE blocks. One block uses *$dumpvars* for RTL instrumentation and the other block uses assertions:

```
device jtagport builtin
device umr_pipe 1
iice new {IICE_DV} -type regular -mode {dumpvars}
device jtagport builtin
```

Feedback

```
device umr_pipe 1
iice new {IICE_SVA} -type regular -mode {sva}
```

2. For *$dumpvars*, use labels to group signals into individual IICEs.
   - Declare multiple IICEs in the IDC file with the **iice new** command.
   - In each RTL *$dumpvars* definition, add a label that corresponds to an IICE name. Use the same label for the *$dumpvars* that you want to group together, and at compile time they are grouped together in the same IICE. See Examples of $dumpvars Labels for Multi-IICE Grouping for examples.

     The following snippet adds *$dumpvars* labels called *iice_bram* and *iice_dtd*:

     ```
     initial
     begin: iice_bram
     $dumpvars(1,top.inst1.sig1,top.inst2.sig2,top.inst3.sig3);
     end

     initial
     begin: iice_dtd
     $dumpvars(1,top.inst4.sig1,top.inst5.sig2,top.inst6.sig3);
     end
     ```

     The IDC file contains matching IICE names. This ensures that the signals from the corresponding *$dumpvars* are routed to the IICE with the matching name.

     ```
     device jtagport builtin
     iice new {iice_bram} -type regular
     device jtagport builtin
     iice new {iice_dtd} -type regular
     ```

Examples of $dumpvars Labels for Multi-IICE Grouping

This section contains examples of multiple IICE units with *$dumpvars* label-based grouping. The examples illustrate the methods to declare multiple IICEs in the IDC using *$dumpvars* labels in the RTL. They also describe the results in different scenarios.

Multiple IICEs and $dumpvars with the Same IICE and Label Names

This example has two IICEs and two *$dumpvars* definitions with the same label names. In this scenario, the tool routes instrumentation from each *$dumpvars* to the corresponding IICE name defined in the IDC file.

```
initial
begin: iice_bram
$dumpvars(1,top.inst1.sig1,top.inst2.sig2,top.inst3.sig3);
end

initial
begin: iice_dtd
```

Feedback

```
$dumpvars(1,top.inst4.sig1,top.inst5.sig2,top.inst6.sig3);
end

device jtagport builtin
iice new {iice_bram} -type regular
device jtagport builtin
iice new {iice_dtd} -type regular
```

Multiple IICE Units and $dumpvars with Different IICE and Label Names

This example has two IICEs and two *$dumpvars* definitions with different label names. In this scenario, instrumentation from both *$dumpvars* must be routed to each IICE. That is, each IICE must have instrumentation from both the *$dumpvars*.

```
initial
begin: iice_bram
$dumpvars(1,top.inst1.sig1,top.inst2.sig2,top.inst3.sig3);
end

initial
begin: iice_dtd
$dumpvars(1,top.inst4.sig1,top.inst5.sig2,top.inst6.sig3);
end

device jtagport builtin
iice new {iice_0} -type regular -mode {dumpvars}
device jtagport builtin
iice new {iice_1} -type regular -mode {dumpvars}
```

Multiple IICE Units and $dumpvars with Escaped Characters in Names

This example has two IICEs and two *$dumpvars* definitions, where the label and IICE names contain escaped characters. In this case, the instrumentor displays an error, as IICE names cannot have special characters.

```
initial
begin: \i$$e_uc
$dumpvars(1,top.inst1.sig1,top.inst2.sig2,top.inst3.sig3);
end

initial
begin: \i*_ce_sm
$dumpvars(1,top.inst4.sig1,top.inst5.sig2,top.inst6.sig3);
end

device jtagport builtin
iice new {i$$e_uc} -type regular
device jtagport builtin
iice new {i*_ce_sm} -type regular
```

Multiple IICE Units with Same Name, Single $dumpvars

Feedback

This example design contains two IICEs and one *$dumpvars* definition. The IICEs have the same label and names but different configurations. As the IICE names are the same, the second IICE replaces the first IICE in the flow.

```
initial
begin: iice_bram
$dumpvars(1,top.inst1.sig1,top.inst2.sig2,top.inst3.sig3);
end

device jtagport builtin
iice new {iice_bram} -type regular
iice controller -iice {iice_bram} none
iice sampler -iice {iice_bram} -depth 128
iice sampler -iice {iice_bram} -pipe 3

device jtagport builtin
iice new {iice_bram} -type regular
iice controller -iice {iice_bram} none
iice sampler -iice {iice_bram} -depth 1024
iice sampler -iice {iice_bram} -pipe 6
```

Single IICE, Multiple $dumpvars with Name Mismatch

This example design contains one IICE and two *$dumpvars* definitions. The first label name matches the IICE name. The second *$dumpvars* label is different from the IICE name and the first *$dumpvars* label name. In this case, instrumentation from the first *$dumpvars* label must be routed to the IICE while the other *$dumpvars* label is ignored.

```
begin: iice_bram
$dumpvars(1,top.inst1.sig1,top.inst2.sig2,top.inst3.sig3);
end

initial
begin: iice_dtd
$dumpvars(1,top.inst4.sig1,top.inst5.sig2,top.inst6.sig3);
end

device jtagport builtin
iice new {iice_bram} -type regular
```

Multiple IICE Units and $dumpvars: Labels with Different Cases

In this example, the design contains two IICEs and two *$dumpvars* definitions. The label names match the IICE names but have different cases. In this case, each IICE has instrumentation from both the *$dumpvars* because the cases are different. That is, *IICE_BRAM* and *IICE_DTD* must be instrumented with signals coming from both the *$dumpvars* labels.

```
initial
begin: iice_bram
$dumpvars(1,top.inst1.sig1,top.inst2.sig2,top.inst3.sig3);
end
```

Feedback

```
initial
begin: iice_dtd
$dumpvars(1,top.inst4.sig1,top.inst5.sig2,top.inst6.sig3);
end

device jtagport builtin
iice new {IICE_BRAM} -type regular
device jtagport builtin
iice new {IICE_DTD} -type regular
```

Single IICE and $dumpvars: Matching IICE and $dumpvars Instance

In this example, the design contains one IICE and one *$dumpvars* definition with matching label names and IICE names. Additionally, the instance name of the *$dumpvars* definition is the same as the IICE name and label name.

This scenario is not supported, so no *$dumpvars* will be created. It is recommended that you avoid using label names that match the *$dumpvars* instance name, as shown in the example below.

```
initial
begin: top
$dumpvars(1,top);
end

device jtagport builtin
iice new {top} -type regular
```

5

HAPS Deep Trace Debug

HAPS Deep Trace Debug (DTD) provides a mechanism to use memory beyond FPGA memory as storage for debug samples. Using external memory makes it possible to have a deeper storage buffer with more capacity for the debug signal data.

HAPS systems include built-in DDR4 memory on daughter cards, The daughter cards are installed by default on each FPGA in a HAPS system. You can use the DDR memory to capture debug samples. This expands storage capacity and makes larger visibility windows possible by using external memory.

The following topic describes how to use DTD with various HAPS systems.

- Specifying DTD Buffers for HAPS Multi-FPGA Designs

Specifying DTD Buffers for HAPS Multi-FPGA Designs

The following procedure describes how to use the built-in DDR4 memory on HAPS systems as DTD buffer memory. Use this methodology for multi-FPGA designs.

The multi-FPGA flow for DTD buffers is similar to the flow with BRAM buffers. But it differs in the maximum number of instrumented bits used per FPGA. There are 18 bits of user bandwidth to transfer through links to assist the FPGA. See the table to determine maximum sampling frequency, based on the number of instrumented bits per FPGA.

1. Specify the DTD buffer to be used for instrumenting the design.

   For multi-FPGA designs, select the DTD buffer type in the Instrumentor at the system-level partitioning stage. For single-FPGA designs, select the buffer type at the FPGA level after partitioning.

   - Open the **Edit IICE Settings** dialog box and set the buffer type to **HAPS-200 Built-in DTD Module**.
   - Alternatively, type the equivalent command:

     ```
     iice sampler -iice <iice_name> haps200_DTD_builtin
     ```

   This setting specifies the dedicated debug memory which has a higher capacity and bandwidth. No FPGA connectors are used. All MIG and trigger data is also stored in the dedicated memory to minimize impact on the user FPGA.

2. Set parameters for the DTD buffer.

   Enhanced DTD Performance improves debug speed and memory capacity for DTD operations. Use this feature to capture data at higher speeds and store more debug information. This helps you validate and debug complex designs faster and with better trace visibility.

   Use the following guidelines to optimize performance for HAPS-200:

Instrumented Bits	Max. Sampling Frequency	Max. Sample Depth
32K *(Beta)*	10 MHz	8388606
16K *(Beta)*	20 MHz	16777214
8K	40 MHz	33554430
4K	80 MHz	67108862
2K	160 MHz	134217726
1K	320 MHz	268435454

   Note:

   For IICE instrumentation, the total number of trigger bits is limited to 8192.

   To instrument signals beyond 8192 bits, make sure:

   - You have specified the `% iice sampler -max_width {16384|32768}` command in the IDC file. This option allows to increase the sample width only, not the trigger width.
   - You do not have more than 8k instrumentation bits of *Sample and Trigger*. The *Trigger only* option is not supported and gets converted to *Sample and Trigger*.
   - Trigger bits do not exceed the limit. Configure remaining signals, other than the 8k instrumentation bits specified above, as **Sample Only**.

   Instrumentation beyond 8K sample width requires a Beta license, and timing closure is not guaranteed due to potential routing congestion.

   Based on the selected buffer type and configuration settings, the sample depth and source clock are calculated. If you do not specify otherwise, the default depth is set at 256. Simple triggering is set for the controller mode for DTD. At runtime, the configured sample depth can be dynamically edited.

3. Run the design through the usual partition flow.

4. Launch individual FPGA runs with place and route, and generate FPGA bit files.

   At runtime, you can use the instrumentation and storage for debug.